

Analysis of Integer Square Root Algorithms in Solidity*

Christopher H. Gorman 

August 29, 2026

Abstract

We investigate methods for computing integer square roots within the Ethereum blockchain while developing new methods that are provably correct. After determining the cost of each algorithm at various inputs, we look at the summary statistics of each method to determine which procedure is most efficient.

1 Introduction

Basic integer arithmetic and bit operations are included within the Ethereum blockchain [37]; all other functions must be derived from those simple operations. We will look at various algorithms for computing integer square roots on the Ethereum blockchain (using the Solidity programming language [31]) and compare their efficiency. This work extends [13] by adding candidate algorithms and improving the analysis. The emphasis is on *provably correct* algorithms which are efficient. A version of these results may be found online [12].

2 Mathematical Definition of Integer Square Root

Given $a \in \mathbb{N}$, the *integer square root* of a is the unique integer m such that

$$m^2 \leq a < (m + 1)^2. \quad (2.1)$$

This will be denoted as

$$\text{ISQRT}(a) := m \quad (2.2)$$

and is equivalent to

$$\text{ISQRT}(a) = \lfloor \sqrt{a} \rfloor, \quad (2.3)$$

the integer part of the square root of a .

*Permanent ID of this document: 021e7116950e28e158ce35151b0dda3a. Date: 2026.08.29.

EVM Ops	Gas
$+, -$	3
$\times, \div$	5
and, or, xor, not	3
clz	5
$<, >, =$	3
$\gg, \ll$	3
Transaction	21 000
sstore (clean)	20 000
sstore (dirty)	5000
Gas Target	15 000 000
Gas Limit	30 000 000

Table 1: Here are some of the gas costs for certain integer and bit operations within the Ethereum Virtual Machine [23, 34, 37]. “Transaction” is the base gas cost of each transaction. “sstore (clean)” refers to writing a nonzero word (32 bytes) to the blockchain; “sstore (dirty)” refers to overwriting a nonzero word with another nonzero word. “Gas Target” is the target amount of gas included within a single Ethereum block; “Gas Limit” is the maximum amount of gas an Ethereum block may contain.

3 Ethereum and the Ethereum Virtual Machine

Ethereum is a blockchain designed to have the ability to perform arbitrary computations [37]. The Ethereum Virtual Machine (EVM) executes all transactions on Ethereum. While Turing-complete, all operations cost a certain amount of *gas* and there is a limit to the amount of gas included within each block. Thus, the efficiency of all operations on Ethereum are measured by their gas usage.

The fundamental type is a `uint256` object: a 256-bit unsigned integer. Table 1 lists the costs of various integer and bit operations within the EVM. The fact that basic integer and bit operations are around 5 gas while every transaction has a base cost of 21 000 gas means that integer and bit operations are inexpensive¹. The fact that clean (dirty) writes costs 20 000 (5000) gas means that storing data on Ethereum is significantly more expensive; thus, it is beneficial to minimize storage. Another notable feature is that multiplication and division have the same cost; this fact is nontrivial, and we note the following from a paper on computing integer square roots: “The absence of division primitives is the main reason why we look for an approximation of the inverse square root rather than the square root itself” [19, Section 2].

4 Generic Algorithm Based on Newton’s Method

One method for computing integer square roots is based on Newton’s method; see [5, Algorithm 1.7.1], [6, Algorithm 9.2.11], and [4, Algorithm 1.13]. While not the only method

¹The recent inclusion of the *count leading zeros* (clz) opcode in EIP-7939 significantly reduces gas costs associated with initializing square roots; this application is specifically mentioned [34, Motivation].

Algorithm 1 Integer Square Root algorithm found in [5, Algorithm 1.7.1]. We include the following from [5, Chapter 1.7, Remarks]: “When actually implementing this algorithm, the initialization step must be modified.”

Require: n is a positive integer

```

1: procedure INTEGERSQUAREROOT( $n$ )
2:    $x := n$ 
3:    $y := \lfloor (x + n/x) / 2 \rfloor$  ▷ All operations are integer operations
4:   while  $x > y$  do
5:      $x := y$ 
6:      $y := \lfloor (x + n/x) / 2 \rfloor$ 
7:   end while
8:   return  $x$ 
9: end procedure

```

for computing integer square roots, the efficiency of Newton’s method causes us to focus on these algorithms; additional algorithms are discussed in Appendix D. These algorithms usually come with a proof of correctness. For completeness, we reproduce [5, Algorithm 1.7.1] in Algorithm 1, and include a modified version of the proof [5, Algorithm 1.7.1, Proof] in Appendix E.5.1.

5 Integer Square Root Algorithms

5.1 Algorithm Design Principle

The algorithm design philosophy is straightforward: provably correct algorithms which are efficient within the Ethereum Virtual Machine. The primary focus is on minimizing the per-call gas cost, but we want a low deployment cost as well.

5.2 Specific Algorithms in Solidity

5.2.1 Online Algorithms

The specific ISQRT algorithms we found online were `UniswapV2` (Listing 1) [33], `PRB` (Listing 2) [2], `PRBv2` (Listing 3) [3], `OpenZeppelin` (Listing 4) [25], `ABDK` (Listing 5) [1], and `OpenZeppelinV2` (Listing 6) [26]. Two algorithms (`PRB` and `ABDK`) appear to be exactly the same aside from different integer notation and “unchecked” portions. The `OpenZeppelin` algorithm is mathematically equivalent to `PRB` and `ABDK`. `OpenZeppelinV2` was developed with input from the author, although in the end his contribution was merely the initialization value that led to needing only six Newton iterations; the author did not write any of the code or comments [7]. `PRBv2` was developed later and uses Solidity assembly language `Yul` [32]².

None of these algorithms reference a proof of correctness except `OpenZeppelinV2` (which was developed after the initial version of the present work). We note that `UniswapV2` is

²At the time of this writing the code has not been merged in, but the code led the author of the present work to rewrite the algorithms in assembly.

Listing 1: Method for computing Integer Square Roots from UniswapV2 [33]

```
// License: GPL-3.0
function sqrt(uint y) internal pure returns (uint z) {
    if (y > 3) {
        z = y;
        uint x = y / 2 + 1;
        while (x < z) {
            z = x;
            x = (y / x + x) / 2;
        }
    } else if (y != 0) {
        z = 1;
    }
}
```

provably correct as it essentially implements [5, Algorithm 1.7.1]; compare Listing 1 and Algorithm 1. Unfortunately, UniswapV2 does not follow the comment in [5, Chapter 1.7, Remarks] that states that “[w]hen actually implementing this algorithm, the initialization step must be modified”. OpenZeppelin [25] says that we “need at most 7 iteration[s]” while ABDK [1] states that “[s]even iterations should be enough”; PRB [2] has similar language. It must be noted that PRBv2 is mathematically equivalent to Unrolled1 and so is provably correct.

5.2.2 Provably Correct Algorithms

Due to requiring *provably correct* methods for computing integer square roots, additional algorithms were written. All proofs are relegated to Appendix E. All of the algorithms discussed here have been written in Yul [32] for efficiency. One important feature is the use of the `clz` function for *count leading zeros*; this was added in Ethereum Improvement Proposal 7939 (EIP-7939) [34]. Using `clz` allows for significant efficiency gains: the new opcode costs 5 gas while the most efficient implementation of the function is estimated to cost 184 gas [34, Specification].

One important application of `clz` is to compute the `bitlength`³ via

$$\text{bitlength}(x) = 256 - \text{clz}(x), \quad x \neq 0. \quad (5.1)$$

The case $x = 0$ must be handled separately, but this is mainly because for $x > 0$ and $\text{bitlength}(x) = k$ we have

$$2^{k-1} \leq x < 2^k. \quad (5.2)$$

Therefore, while Eq. (5.1) gives $\text{bitlength}(0) = 0$, this does not result in an inequality similar to Eq. (5.2).

³Thus, we expect initializations which compute the bit length *without* using `clz` will cost at least 184 gas.

Listing 2: Method for computing Integer Square Roots from PRB [2]

```
// SPDX-License-Identifier: MIT
function sqrt(uint256 x) internal pure returns (uint256 result) {
    if (x == 0) { return 0; }
    uint256 xAux = uint256(x);
    result = 1;
    if (xAux >= 2 ** 128) {
        xAux >>= 128;    result <<= 64;
    }
    if (xAux >= 2 ** 64) {
        xAux >>= 64;    result <<= 32;
    }
    if (xAux >= 2 ** 32) {
        xAux >>= 32;    result <<= 16;
    }
    if (xAux >= 2 ** 16) {
        xAux >>= 16;    result <<= 8;
    }
    if (xAux >= 2 ** 8) {
        xAux >>= 8;    result <<= 4;
    }
    if (xAux >= 2 ** 4) {
        xAux >>= 4;    result <<= 2;
    }
    if (xAux >= 2 ** 2) {
        result <<= 1;
    }
    unchecked {
        result = (result + x / result) >> 1;
        result = (result + x / result) >> 1;
        result = (result + x / result) >> 1;
        result = (result + x / result) >> 1;
        result = (result + x / result) >> 1;
        result = (result + x / result) >> 1;
        result = (result + x / result) >> 1;
        uint256 roundedResult = x / result;
        if (result >= roundedResult) {
            result = roundedResult;
        }
    }
}
```

Listing 3: Method for computing Integer Square Roots from PRBv2 [3]

```
// SPDX-License-Identifier: MIT
function sqrt(uint256 x) pure returns (uint256 result) {
    unchecked {
        result = 1 << (msb(x) >> 1);
    }

    assembly ("memory-safe") {
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))

        result := sub(result, gt(result, div(x, result)))
    }
}

function msb(uint256 x) pure returns (uint256 result) {
    assembly ("memory-safe") {
        result := shl(7, lt(0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF, x))
        result := or(result, shl(6, lt(0xFFFFFFFFFFFFFFFF,
                                     shr(result, x))))
        result := or(result, shl(5, lt(0xFFFFFFFF,
                                     shr(result, x))))
        result := or(result, shl(4, lt(0xFFFF, shr(result, x))))
        result := or(result, shl(3, lt(0xFF, shr(result, x))))
        result := or(result, shl(2, lt(0xF, shr(result, x))))
        result := or(result, shl(1, lt(0x3, shr(result, x))))
        result := or(result, lt(0x1, shr(result, x)))
    }
}
```

Listing 4: Method for computing Integer Square Roots from OpenZeppelin [25]

```
// SPDX-License-Identifier: MIT
function sqrt(uint256 a) internal pure returns (uint256) {
    if (a == 0) {
        return 0;
    }
    uint256 result = 1 << (log2(a) >> 1);
    // At this point 'result' is an estimation with one bit of
    // precision. We know the true value is a uint128, since it is
    // the square root of a uint256. Newton's method converges
    // quadratically (precision doubles at every iteration). We
    // thus need at most 7 iteration to turn our partial result
    // with one bit of precision into the expected uint128 result.
    unchecked {
        result = (result + a / result) >> 1;
        result = (result + a / result) >> 1;
        result = (result + a / result) >> 1;
        result = (result + a / result) >> 1;
        result = (result + a / result) >> 1;
        result = (result + a / result) >> 1;
        result = (result + a / result) >> 1;
        return min(result, a / result);
    }
}

function log2(uint256 value) internal pure returns (uint256) {
    uint256 result = 0;
    unchecked {
        if (value >> 128 > 0) { value >>= 128; result += 128; }
        if (value >> 64 > 0) { value >>= 64; result += 64; }
        if (value >> 32 > 0) { value >>= 32; result += 32; }
        if (value >> 16 > 0) { value >>= 16; result += 16; }
        if (value >> 8 > 0) { value >>= 8; result += 8; }
        if (value >> 4 > 0) { value >>= 4; result += 4; }
        if (value >> 2 > 0) { value >>= 2; result += 2; }
        if (value >> 1 > 0) { result += 1; }
    }
    return result;
}

function min(uint256 a, uint256 b) internal pure returns (uint256){
    return a < b ? a : b;
}
```

Listing 5: Method for computing Integer Square Roots from ABDK [1]

```
// SPDX-License-Identifier: BSD-4-Clause
function sqrt(uint256 x) private pure returns (uint128) {
    unchecked {
        if (x == 0) return 0;
        else {
            uint256 xx = x;
            uint256 r = 1;
            if (xx >= 0x100000000000000000000000000000000) {
                xx >>= 128; r <<= 64;
            }
            if (xx >= 0x1000000000000000000000000) {
                xx >>= 64; r <<= 32;
            }
            if (xx >= 0x1000000000) {
                xx >>= 32; r <<= 16;
            }
            if (xx >= 0x10000) {
                xx >>= 16; r <<= 8;
            }
            if (xx >= 0x100) {
                xx >>= 8; r <<= 4;
            }
            if (xx >= 0x10) {
                xx >>= 4; r <<= 2;
            }
            if (xx >= 0x4) {
                r <<= 1;
            }
            r = (r + x / r) >> 1;
            r = (r + x / r) >> 1;
            r = (r + x / r) >> 1;
            r = (r + x / r) >> 1;
            r = (r + x / r) >> 1;
            r = (r + x / r) >> 1;
            r = (r + x / r) >> 1;
            // Seven iterations should be enough
            uint256 r1 = x / r;
            return uint128 (r < r1 ? r : r1);
        }
    }
}
```

Listing 6: Provably correct method for computing Integer Square Roots from OpenZep-
 pelinV2 [26]. This method is based on Unrolled3 (Listing 9).

```
// SPDX-License-Identifier: MIT
function sqrt(uint256 a) internal pure returns (uint256) {
  unchecked {
    if (a <= 1) {
      return a;
    }

    uint256 aa = a;
    uint256 xn = 1;

    if (aa >= (1 << 128)) { aa >>= 128; xn <<= 64; }
    if (aa >= (1 << 64)) { aa >>= 64; xn <<= 32; }
    if (aa >= (1 << 32)) { aa >>= 32; xn <<= 16; }
    if (aa >= (1 << 16)) { aa >>= 16; xn <<= 8; }
    if (aa >= (1 << 8)) { aa >>= 8; xn <<= 4; }
    if (aa >= (1 << 4)) { aa >>= 4; xn <<= 2; }
    if (aa >= (1 << 2)) { xn <<= 1; }

    xn = (3 * xn) >> 1;

    xn = (xn + a / xn) >> 1;
    xn = (xn + a / xn) >> 1;
    xn = (xn + a / xn) >> 1;
    xn = (xn + a / xn) >> 1;
    xn = (xn + a / xn) >> 1;
    xn = (xn + a / xn) >> 1;

    return xn - SafeCast.toUint(xn > a / xn);
  }
}
```

The first algorithms come in two groups of 3. They are `Unrolled1` (Listing 7), `Unrolled2` (Listing 8), `Unrolled3` (Listing 9), `While1` (Listing 10), `While2` (Listing 11), and `While3` (Listing 12). A version of `Unrolled3` was previously presented in [13]; the *initialization value* is the same but the *initialization method* has been updated. Here, `Unrolled` refers to the fact that a fixed number of Newton iterations are performed; `While` refers to the fact that the Newton iterations are iterated within a `while` loop. Labels 1, 2, and 3 refer to the different initialization values: the largest power-of-two less than or equal to the integer square root value; the smallest power-of-two greater than the integer square root value; the arithmetic mean of the two previous values.

Additional algorithms include `BitLength` (Listing 13), `Linear` (Listing 14), `Hyper4` (Listing 15), `Lookup4` (Listing 16), and `Lookup8` (Listing 17). `BitLength` uses the full bitlength of the number to produce an initial approximation. `Linear` uses a linear approximation based on the high bits of the input value. `Hyper4` uses a hyperbolic approximation based on the high bits of the input value; we investigated such approximations based on the discussion in [21]. `Lookup4` and `Lookup8` use lookup tables to obtain 4 bits and 8 bits of accuracy for the initial approximation; the idea to use a lookup table for initialization came from [19, 20], although [19] uses a different method for computing square roots (it computes square roots by first computing the *inverse square root*).

5.2.3 Handling $x = 0$ within the New Algorithms

The `bitlength` function in Eq. (5.1) is modified in the new algorithms for efficiency purposes. Even so, the modifications have been proven correct in Appendix E.2.

5.3 General Algorithm Discussion

When using Newton iterations, a good initial approximation is critical to ensure a low overall cost. The following algorithms use the largest power-of-two less than or equal to the integer square root: `PRB` (Listing 2), `PRBv2` (Listing 3), `OpenZeppelin` (Listing 4), `ABDK` (Listing 5), `Unrolled1` (Listing 7), and `While1` (Listing 10). Another good initialization value is shared by `Unrolled2` (Listing 8) and `While2` (Listing 11), which follow the initialization suggestions of [5, Chapter 1.7, Remarks (2)] and [6, Algorithm 9.2.11]. `Unrolled3` (Listing 9), `While3` (Listing 12), and `OpenZeppelinV2` (Listing 6) use the new initialization from [13]. `BitLength` (Listing 13), `Linear` (Listing 14), `Hyper4` (Listing 15), `Lookup4` (Listing 16), and `Lookup8` (Listing 17) use initializations specific to each method. `UniswapV2` (Listing 1) has the worst initialization value (the input value itself).

6 Comparison of Gas Costs

Because all operations are performed within the EVM, the primary cost metric is the overall gas cost. Thus, the focus is on minimizing the total gas used. Ideally, the best algorithm will have the lowest maximum, mean, and median gas costs. We note that the results here are different from those in [13] for three reasons: first, some of the algorithms are different (and we chose the most recent version of those found online); second, we ensure that all input

Listing 7: Provably correct method for computing Integer Square Roots (Unrolled1)

```
// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 result) {
    assembly ("memory-safe") {
        result := clz(x)
        result := sub(255, result)
        // If
        //
        //      2**(k-1) <= x < 2**k
        //
        // we now have
        //
        //      result == k-1

        result := shl(shr(1, result), 1)
        // If
        //
        //      2**(f-1) <= sqrt(x) < 2**f
        //
        // we now have
        //
        //      result == 2**(f-1)

        // Perform the 7 required newton iterations
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))

        result := sub(result, gt(result, div(x, result)))
    }
}
```

Listing 8: Provably correct method for computing Integer Square Roots (Unrolled2)

```
// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 result) {
    assembly ("memory-safe") {
        result := clz(x)
        result := sub(257, result)
        // If
        //
        //      2**(k-1) <= x < 2**k
        //
        // we now have
        //
        //      result == k-1

        result := shl(shr(1, result), 1)
        // If
        //
        //      2**(f-1) <= sqrt(x) < 2**f
        //
        // we now have
        //
        //      result == 2**f

        // Perform the 7 required newton iterations
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))

        result := sub(result, gt(result, div(x, result)))
    }
}
```

Listing 9: Provably correct method for computing Integer Square Roots (Unrolled3)

```
// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 result) {
    assembly ("memory-safe") {
        result := clz(x)
        result := sub(255, result)
        // If
        //
        //      2**(k-1) <= x < 2**k
        //
        // we now have
        //
        //      result == k-1

        result := shr(1, shl(shr(1, result), 3))
        // If
        //
        //      2**(f-1) <= sqrt(x) < 2**f
        //
        // we now have
        //
        //      result == 2**(f-1) + 2**(f-2)

        // Perform the 6 required newton iterations
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))

        result := sub(result, gt(result, div(x, result)))
    }
}
```

Listing 10: Provably correct method for computing Integer Square Roots (While1)

```
// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 result) {
    assembly ("memory-safe") {
        result := clz(x)
        result := sub(255, result)
        // If
        //
        //      2**(k-1) <= x < 2**k
        //
        // we now have
        //
        //      result == k-1

        result := shl(shr(1, result), 1)
        // If
        //
        //      2**(f-1) <= sqrt(x) < 2**f
        //
        // we now have
        //
        //      result == 2**(f-1)

        result := shr(1, add(result, div(x, result)))
        let xAux := shr(1, add(result, div(x, result)))

        // while loop
        for { } lt(xAux, result) { } {
            result := xAux
            xAux := shr(1, add(result, div(x, result)))
        }
    }
}
```

Listing 11: Provably correct method for computing Integer Square Roots (While2)

```
// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 result) {
    assembly ("memory-safe") {
        result := clz(x)
        result := sub(257, result)
        // If
        //
        //      2**(k-1) <= x < 2**k
        //
        // we now have
        //
        //      result == k-1

        result := shl(shr(1, result), 1)
        // If
        //
        //      2**(f-1) <= sqrt(x) < 2**f
        //
        // we now have
        //
        //      result == 2**f

        let xAux := shr(1, add(result, div(x, result)))

        // while loop
        for { } lt(xAux, result) { } {
            result := xAux
            xAux := shr(1, add(result, div(x, result)))
        }
    }
}
```

Listing 12: Provably correct method for computing Integer Square Roots (While3)

```
// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 result) {
    assembly ("memory-safe") {
        result := clz(x)
        result := sub(255, result)
        // If
        //
        //      2**(k-1) <= x < 2**k
        //
        // we now have
        //
        //      result == k-1

        result := shr(1, shl(shr(1, result), 3))
        // If
        //
        //      2**(f-1) <= sqrt(x) < 2**f
        //
        // we now have
        //
        //      result == 2**(f-1) + 2**(f-2)

        result := shr(1, add(result, div(x, result)))
        let xAux := shr(1, add(result, div(x, result)))

        // while loop
        for { } lt(xAux, result) { } {
            result := xAux
            xAux := shr(1, add(result, div(x, result)))
        }
    }
}
```

Listing 13: Provably correct method for computing Integer Square Roots (BitLength)

```
// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 result) {
    assembly ("memory-safe") {
        result := clz(x)
        result := sub(255, result)
        // If
        //
        //      2**(k-1) <= x < 2**k
        //
        // we now have
        //
        //      result == k-1
        //
        // Thus, we actually are using the bitlength minus one

        switch and(result, 1)
        case 1 {
            // bitlength is *even*
            result := shr(4, shl(shr(1, result), 27))
        }
        default {
            // bitlength is *odd*
            result := shr(5, shl(shr(1, result), 39))
        }

        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))
        result := shr(1, add(result, div(x, result)))

        result := sub(result, gt(result, div(x, result)))
    }
}
```

Listing 14: Provably correct method for computing Integer Square Roots (Linear)

```
// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 result) {
    assembly ("memory-safe") {
        let e := clz(x)
        e := shr(1, e)
        let m := shl(shl(1, e), x)

        result := shr(252, m)
        // the initial approximation
        result := shl(123, add(14, result))

        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(e, result)

        result := sub(result, gt(result, div(x, result)))
    }
}
```

Listing 15: Provably correct method for computing Integer Square Roots (Hyper4)

```
// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 result) {
    assembly ("memory-safe") {
        let e := clz(x)
        e := shr(1, e)
        let m := shl(shl(1, e), x)

        result := shr(252, m)
        // the initial approximation
        result := shl(123, div(512, sub(31, result)))

        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(e, result)

        result := sub(result, gt(result, div(x, result)))
    }
}
```

Listing 16: Provably-correct method for computing Integer Square Roots (Lookup4)

```
// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 result) {
    assembly ("memory-safe") {
        let e := clz(x)
        e := shr(1, e)
        let m := shl(shl(1, e), x)

        // the initial approximation
        result := shr(252, m) // get the top 4 bits for lookup
        result := byte(result,
            0x0000000001113151617191a1b1c1d1e1f
            00000000000000000000000000000000)
        result := shl(123, result)

        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(e, result)

        result := sub(result, gt(result, div(x, result)))
    }
}
```

Listing 17: Provably-correct method for computing Integer Square Roots (Lookup8)

```
// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 result) {
    bytes memory lookup_table = lookup_table_8;
    assembly ("memory-safe") {
        // shift past the first 32 bytes
        // (as these store the array length)
        let data_ptr := add(lookup_table, 0x20)

        let e := clz(x)
        e := shr(1, e)
        let m := shl(shl(1, e), x)

        // the initial approximation
        result := shr(248, m) // get top 8 bits for index
        let word_ptr := add(data_ptr, result) // shift by index
        let word := mload(word_ptr) // grab 32-byte word
        result := byte(0, word) // get lookup value
        result := add(256, result) // add implicit high bit
        result := shl(119, result) // shift

        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(1, add(result, div(m, result)))
        result := shr(e, result)

        result := sub(result, gt(result, div(x, result)))
    }
}

bytes constant lookup_table_8 = "...";
```

values are unique (certain values may have been double counted); third, different input values were chosen.

After analyzing the per-call gas cost distribution, we discuss the associated costs of deploying the algorithm to the Ethereum blockchain.

6.1 Data Point Selection

Any gas cost comparison requires the selection of `uint256` values for input. The values were chosen in this way:

- all numbers of the form $2^k - 1$, 2^k , and $2^k + 1$ for positive values of k ;
- $v - 1$, v , and $v + 1$ for $v = (2^{128} - 1)^2$ (these were chosen because of a previous edge case in some algorithms);
- random values chosen according to the loguniform distribution [29] on $[1, 2^{256}]$ when the random seed is initialized to 0 [24].

The number of random samples were increased until there were 2048 unique values. While it is clear that the particular distribution affects the results, we believe choosing the input values in this way reduces the risk of bias. There were a total of 768 deterministic values chosen and a total of 1303 random samples determined the remaining 1280 values.

In addition to measuring the gas cost, the result of each integer square root operation was validated using Python’s integer square root [8, 27]. There was no instance in which an algorithm ever produced an incorrect result; this is, of course, not a proof that PRB, OpenZeppelin, and ABDK are correct, but rather that there are no known values where these algorithms fail.

6.2 Summary Statistics

A listing of the summary statistics describing the runs may be found in Tables 2, 3, and 4. From here, we see that the algorithm with the smallest mean, median, and maximum gas cost is `Unrolled3`; the standard deviation of `Unrolled3` is minimal as well. `Unrolled1` and `Unrolled2` tied for second place followed by `Linear` in fourth.

6.3 Detailed Analysis

We now take a closer look at how the gas cost varies with the argument.

For each value we tested, we determined the minimal gas cost and then counted the number of times each algorithm was minimal; the results may be found in Table 5. Additionally, we show a histogram where certain algorithms were minimal in Figure 1. There does not appear to be a particular pattern in the value distribution. The main trend is that `Unrolled3` consistently performs well across the range of arguments. The only region where this does not hold is for small arguments, where other algorithms (`While`) performed better; this makes sense because those values require fewer Newton iterations.

	UniswapV2	PRB	PRBv2	OpenZeppelin	ABDK	OpenZeppelinV2
Max	33 931	874	474	1015	877	823
Mean	17 591	791	474	944	798	749
Median	17 497	794	474	943	799	751
Std	9482	34	0	30	33	35
Abs Cost	224 112	275 009	216 988	278 189	279 161	268 989
Rel Cost	44 137	95 034	37 013	98 214	99 186	89 014

Table 2: Here are statistics related to the gas cost data; we also include the absolute deployment gas cost and relative deployment gas cost (absolute gas cost less 179 975 gas, the deployment cost of an empty smart contract). The `UniswapV2`, `PRBv2`, and `OpenZeppelinV2` algorithms are provably correct. These results are for the tests in Section 6.

	Unrolled1	Unrolled2	Unrolled3	While1	While2	While3
Max	275	275	269	677	657	626
Mean	275	275	269	448	503	459
Median	275	275	269	482	527	496
Std	0	0	0	119	102	85
Abs Cost	194 259	194 475	194 043	195 075	193 995	196 155
Rel Cost	14 284	14 500	14 068	15 100	14 020	16 180

Table 3: Here are statistics related to the gas cost data; we also include the absolute deployment gas cost and relative deployment gas cost (absolute gas cost less 179 975 gas, the deployment cost of an empty smart contract). All of these algorithms are provably correct. These results are for the tests in Section 6.

	BitLength	Linear	Hyper4	Lookup4	Lookup8
Max	349	279	287	285	440
Mean	344	279	287	285	440
Median	339	279	287	285	440
Std	5	0	0	0	0
Abs Cost	203 963	202 055	202 907	208 172	268 281
Rel Cost	23 988	22 080	22 932	28 197	88 306

Table 4: Here are statistics related to the gas cost data; we also include the absolute deployment gas cost and relative deployment gas cost (absolute gas cost less 179 975 gas, the deployment cost of an empty smart contract). All of these algorithms are provably correct. These results are for the tests in Section 6.

Total	2048
UniswapV2	2
OpenZeppelinV2	2
Unrolled3	1582
While1	282
While2	134
While3	46

Table 5: Here are the number of times each method had minimal gas cost; methods not included were never minimal. These results are for the tests in Section 6.

Overall, **Unrolled3** has the lowest cost for a majority of points, so we declare this to be the algorithm of choice. An extended analysis (with similar results) may be found in Appendix A.

6.4 Deployment Gas Cost

While the primary design criterion is provably-correct algorithm with efficient operation, it is important to recognize the initial (one-time) cost of *deploying* the algorithm to Ethereum.

The deployment cost is related to the Ethereum bytecode; this is the compiled version of the algorithm which runs on the Ethereum Virtual Machine. The cost primarily depends on the size of the bytecode.

In Tables 2, 3, and 4, we include the absolute and relative deployment costs: the absolute deployment cost is the gas cost given by the Hardhat Solidity compiler [10]⁴; the relative deployment cost is obtained by subtracting the deployment cost of an empty smart contract (179 975 gas). We highlight algorithms with relative deployment cost below 20 000 gas; this value was chosen somewhat arbitrarily but makes it easier to distinguish algorithms by deployment cost⁵.

Because the cost of deployment is a one-time cost, the data here is included to provide a larger picture. The results show the lowest deployment costs are in the order **While2**, **Unrolled3**, **Unrolled1**, and **Unrolled2**, although the cost differences are minimal.

7 Conclusion

After comparing various algorithms for total gas cost, we found that the most efficient algorithm based on the sampled values is **Unrolled3** (Listing 9); it also has the second-lowest deployment cost. A proof of correctness may be found Appendix E.

In Appendix B.1, we see that the different initializations change the required number of iterations but not its asymptotic nature; that is, computing the integer square root using

⁴Using Solidity version 0.8.31 and the optimizer enabled for 1 000 000 runs.

⁵Using the Yul assembly language [32] to write the new algorithms resulted in a *significant* decrease in deployment cost. Previously, the minimal deployment cost was **UniswapV2** and the minimal *unrolled* cost was **OpenZeppelinV2** (with roughly the same costs as shown in Table 2); now, essentially all of the new algorithms have smaller cost than **UniswapV2**.

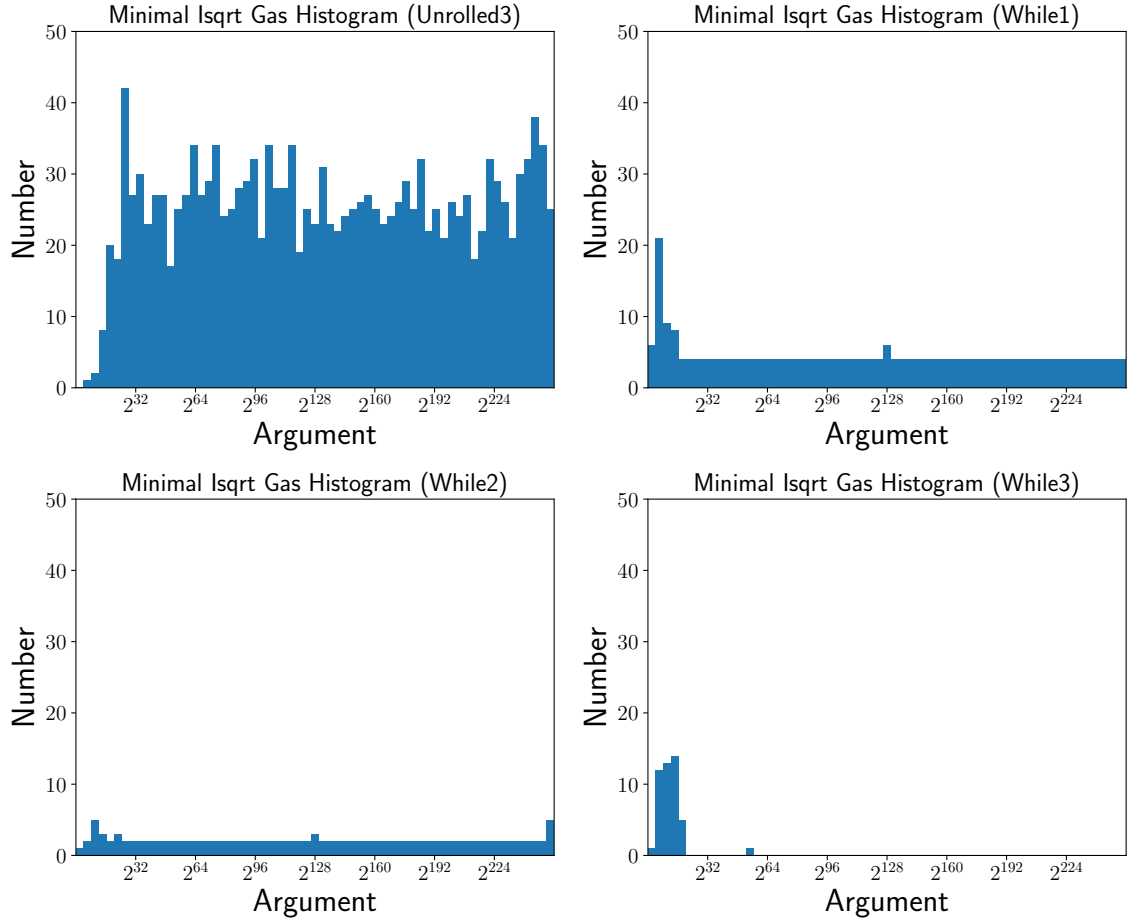


Figure 1: Here we plot a histogram showing where each algorithm is minimal and provides more detail to the results in Table 5. Each bin counts the total number of instances where the algorithm's gas cost is minimal. These results are for the tests in Section 6.

Newton iteration requires $O(\lg \lg n)$ iterations⁶. In Appendix F, we work through the details of algorithms which use *one* floating-point square root as part of the computation (as suggested by [5, Chapter 1.7, Remarks]). It should be noted that any *significant* computation should use a dedicated library for these computations such as GMP [14] or Flint [9].

References

- [1] ABDK Consulting. *Integer Square Root Algorithm in Solidity*. Oct. 2020. URL: <https://github.com/abdk-consulting/abdk-libraries-solidity/blob/9e69beb255e2f87d67b44047daeABDKMath64x64.sol#L727> (cit. on pp. 3, 4, 8, 63).
- [2] Paul Razvan Berg. *Integer Square Root Algorithm in Solidity*. June 2023. URL: <https://github.com/PaulRBerg/prb-math/blob/28055f6cd9a2367f9ad7ab6c8e01c9ac8e9acc61/src/Common.sol#L595> (cit. on pp. 3–5, 63).
- [3] Paul Razvan Berg and Tiago de Paula. *Integer Square Root Algorithm in Solidity*. July 2025. URL: <https://github.com/PaulRBerg/prb-math/pull/264> (cit. on pp. 3, 6).
- [4] Richard P. Brent and Paul Zimmermann. *Modern Computer Arithmetic*. Version 0.5.1. 2010. DOI: <https://doi.org/10.48550/arXiv.1004.4710> (cit. on pp. 2, 63).
- [5] Henri Cohen. *A Course in Computational Algebraic Number Theory*. Graduate Texts in Mathematics 138. Springer, 1993, pp. xx+545. ISBN: 3-540-55640-0 (cit. on pp. 2–4, 10, 26, 63, 64, 67, 68).
- [6] Richard Crandall and Carl Pomerance. *Prime Numbers: A Computational Perspective*. 2nd ed. Springer, 2005, pp. xv+597. ISBN: 978-0387-25282-7 (cit. on pp. 2, 10, 63).
- [7] Hadrien Croubois, Ernesto García, and Christopher H. Gorman. *Improved integer square root (#4403)*. Feb. 16, 2024. URL: <https://github.com/OpenZeppelin/openzeppelin-contracts/commit/140d66fad8b3d8bf1fe793fc73d5e0a9f55f16e3> (visited on 12/11/2024) (cit. on p. 3).
- [8] Mark Dickinson. *Python’s integer square root algorithm*. Jan. 2022. URL: <https://github.com/mdickinson/snippets/blob/master/papers/isqrt/isqrt.pdf> (cit. on pp. 22, 39, 40, 63).
- [9] The FLINT team. *FLINT: Fast Library for Number Theory*. Version 3.4.0, <https://flintlib.org>. 2025 (cit. on pp. 26, 67, 73).
- [10] Nomic Foundation. *Hardhat*. Version 3.12.0. July 30, 2026. URL: <https://hardhat.org/> (cit. on p. 24).
- [11] Gaston H. Gonnet, Lawrence D. Rogers, and J. Alan George. “An Algorithmic and Complexity Analysis of Interpolation Search”. In: *Acta Informatica* 13 (1980), pp. 39–52. URL: https://www.researchgate.net/profile/Gaston-Gonnet/publication/220197937_An_Algorithmic_and_Complexity_Analysis_of_Interpolation_Search/links/00b7d5327f9304caef000000/An-Algorithmic-and-Complexity-Analysis-of-Interpolation-Search.pdf (cit. on pp. 42, 44).

⁶Thus, improved initializations give *practical* but not *theoretical* improvements.

- [12] Christopher H. Gorman. *Analysis of Integer Square Root Algorithms in Solidity*. 2023. URL: <https://github.com/chgorman/isqrt-gas> (cit. on p. 1).
- [13] Christopher H. Gorman. *Efficient Integer Square Roots in Solidity*. Nov. 2022. URL: https://github.com/alicenet/.github/blob/legacy/1.0/docs/efficient_isqrt.pdf (cit. on pp. 1, 10, 39, 40, 47).
- [14] Torbjörn Granlund and the GMP development team. *GNU MP: The GNU Multiple Precision Arithmetic Library*. 6.3.0. 2023. URL: <http://gmplib.org/> (cit. on pp. 26, 67, 73).
- [15] Chun-Hua Guo. “On Newton’s method and Halley’s method for the principal p th root of a matrix”. In: *Linear Algebra and its Applications* 432.8 (2010), pp. 1905–1922. DOI: <https://doi.org/10.1016/j.laa.2009.02.030> (cit. on p. 33).
- [16] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Second. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics, 2002, pp. xxx+680. ISBN: 0-89871-521-0 (cit. on p. 67).
- [17] Donald Ervin Knuth. *The Art of Computer Programming, Volume 2, Seminumerical Algorithms*. 3rd ed. Addison-Wesley, Reading, MA, USA, 1998, pp. xiii+762. ISBN: 0-201-89684-2 (cit. on p. 67).
- [18] Donald Ervin Knuth. *The Art of Computer Programming, Volume 3, Sorting and Searching*. 2nd ed. Addison-Wesley, Reading, MA, USA, 1998, pp. xiii+782. ISBN: 0-201-89685-0 (cit. on p. 39).
- [19] Guillaume Melquiond and Raphaël Rieu-Helft. “Formal Verification of a State-of-the-Art Integer Square Root”. In: *2019 IEEE 26th Symposium on Computer Arithmetic (ARITH)*. IEEE. 2019, pp. 183–186. DOI: <https://doi.org/10.1109/ARITH.2019.00041>. URL: <https://inria.hal.science/hal-02092970/file/main.pdf> (cit. on pp. 2, 10, 39).
- [20] *Methods of computing square roots: Binary estimates*. URL: https://en.wikipedia.org/wiki/Methods_of_computing_square_roots#Binary_estimates (visited on 08/14/2023) (cit. on p. 10).
- [21] *Methods of computing square roots: Hyperbolic estimates*. URL: https://en.wikipedia.org/wiki/Methods_of_computing_square_roots#Hyperbolic_estimates (visited on 08/14/2023) (cit. on p. 10).
- [22] Adnan Saher Mohammed, Şahin Emrah Amrahov, and Fatih V Çelebi. “Interpolated binary search: An efficient hybrid search algorithm on ordered datasets”. In: *Engineering Science and Technology, an International Journal* 24.5 (2021), pp. 1072–1079. URL: <https://www.sciencedirect.com/science/article/pii/S221509862100046X> (cit. on p. 44).
- [23] Nero_eth. *On Block Sizes, Gas Limits and Scalability*. URL: <https://ethresear.ch/t/on-block-sizes-gas-limits-and-scalability/18444> (visited on 11/11/2024) (cit. on p. 2).
- [24] Numpy. *numpy.random.seed*. URL: <https://numpy.org/doc/stable/reference/random/generated/numpy.random.seed.html> (visited on 07/16/2023) (cit. on p. 22).

- [25] OpenZeppelin. *Integer Square Root Algorithm in Solidity*. July 2023. URL: <https://github.com/OpenZeppelin/openzeppelin-contracts/blob/6bf68a41d19f3d6e364d8c207cb7d1acontracts/utils/math/Math.sol#L220> (cit. on pp. 3, 4, 7, 63).
- [26] OpenZeppelin. *Integer Square Root Algorithm in Solidity*. Feb. 2024. URL: <https://github.com/OpenZeppelin/openzeppelin-contracts/blob/140d66fad8b3d8bf1fe793fc73d5e0acontracts/utils/math/Math.sol#L391> (cit. on pp. 3, 9).
- [27] Python. *Mathematical Functions*. URL: <https://docs.python.org/3/library/math.html#math.isqrt> (visited on 07/16/2023) (cit. on p. 22).
- [28] Walter Rudin. *Principles of Mathematical Analysis*. 3rd ed. International Series in Pure and Applied Mathematics. McGraw-Hill New York, 1976, pp. viii+342. ISBN: 978-0-07-054235-8 (cit. on pp. 32, 71).
- [29] Scipy. *scipy.stats.loguniform*. URL: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.loguniform.html> (visited on 07/16/2023) (cit. on pp. 22, 29).
- [30] J. Michael Steele. *The Cauchy-Schwarz Master Class: An Introduction to the Art of Mathematical Inequalities*. MAA Problem Books. Cambridge University Press, 2004, pp. x+306. ISBN: 978-0-521-54677-5 (cit. on p. 47).
- [31] Solidity Development Team. *Solidity Language*. URL: <https://soliditylang.org/> (visited on 12/15/2024) (cit. on p. 1).
- [32] Solidity Development Team. *Yul Language*. URL: <https://docs.soliditylang.org/en/latest/yul.html> (visited on 08/13/2026) (cit. on pp. 3, 4, 24, 34).
- [33] Uniswap. *Integer Square Root Algorithm in Solidity*. Jan. 2020. URL: <https://github.com/Uniswap/v2-core/blob/585ee2ef824db671b71e351a91860a003c05431d/contracts/libraries/Math.sol#L11> (cit. on pp. 3, 4).
- [34] Vectorized (@Vectorized), Georgios Konstantopoulos (@gakonst), Jochem Brouwer (@jochem-brouwer), Ben Adams (@benaadams), and Giulio Rebuffo (@Giulio2002). *Ethereum Improvement Proposals, no. 7939*. Apr. 2025. URL: <https://eips.ethereum.org/EIPS/eip-7939> (visited on 08/14/2026) (cit. on pp. 2, 4).
- [35] Henry S. Warren Jr. *Hacker’s Delight*. 2nd ed. Addison-Wesley, 2013, pp. xvi+492. ISBN: 978-0-321-84268-8 (cit. on pp. 39, 42, 44, 46, 63).
- [36] Henry S. Warren Jr. *Hacker’s Delight Integer Square Root Code*. 2013. URL: <https://web.archive.org/web/20190402153025/http://www.hackersdelight.org/hdcodetxt/isqrt.c.txt> (visited on 03/16/2024) (cit. on pp. 42, 44).
- [37] Gavin Wood. *Ethereum: A Secure Decentralised Generalised Transaction Ledger*. Yellow-paper. Ethereum Project, 2022. URL: <https://github.com/ethereum/yellowpaper> (cit. on pp. 1, 2).
- [38] Andrew C. Yao and F. Francis Yao. “The complexity of searching an ordered random table”. In: *2013 IEEE 54th Annual Symposium on Foundations of Computer Science*. Los Alamitos, CA, USA: IEEE Computer Society, Oct. 1976, pp. 173–177. DOI: 10.1109/SFCS.1976.32. URL: <https://doi.ieeecomputersociety.org/10.1109/SFCS.1976.32> (cit. on p. 42).

- [39] Paul Zimmermann. *Karatsuba Square Root*. Research Report RR-3805. INRIA, 1999, p. 8. URL: <https://inria.hal.science/inria-00072854> (cit. on p. 67).

A Extended Analysis

While the results in Section 6 are good, it is desirable to have a more extensive analysis. The only algorithms tested are `Unrolled3` (Listing 9), `While1` (Listing 10), `While2` (Listing 11), and `While3` (Listing 12), as these performed the best: they had a significant number of values which were minimal.

The deterministic and random values will be separated. All values tested in Section 6 are included in these tests.

A.1 Deterministic Tests

We test additional deterministic values. First, we include all deterministic values from Section 6. Additional values are of the form $2^k + 2^j$ for $j, k \in \mathbb{N}$ and $j < k$. The total number of values were 33 154. The results are shown in Table 6 and Figure 2.

The minimal values have the same trend as before: `Unrolled3` performs well. In a previous version of this document (which compared algorithms which were not written in assembly), `Unrolled3` scored only slightly higher than `While1` in this test (because the `While1` initialization values were very close to the correct result); that is no longer the case. See the discussion on gas costs of `while` loops in Appendix C.2.2.

A.2 Loguniform Tests

The tests in Section 6 ran 1280 values which came from a loguniform distribution [29]. Here, we perform the same test using more samples; in particular, we use $16\,384 = 2^{14}$ random values from 16 808 samples. The results are shown in Table 7 and Figure 3. In this case, `Unrolled3` performs well.

There is no significant difference between the results in Tables 5 and 7 or Figures 1 and 3. `Unrolled3` has a majority of the minimal cost for random values.

A.3 Conclusion

This extended analysis gives additional insight but the same result: `Unrolled3` is the best algorithm for a generic `uint256` value.

B Asymptotic Analysis

We investigate the asymptotic error estimates of computing square roots. We focus on determining the number of iterations required based on Newton’s method and its generalizations.

Total	33 154
Unrolled3	24 442
While1	8391
While2	130
While3	191

Table 6: Here are the number of times each method had minimal gas cost; these are results for the additional deterministic values based around powers-of-two. These results are for the tests in Appendix A.1.

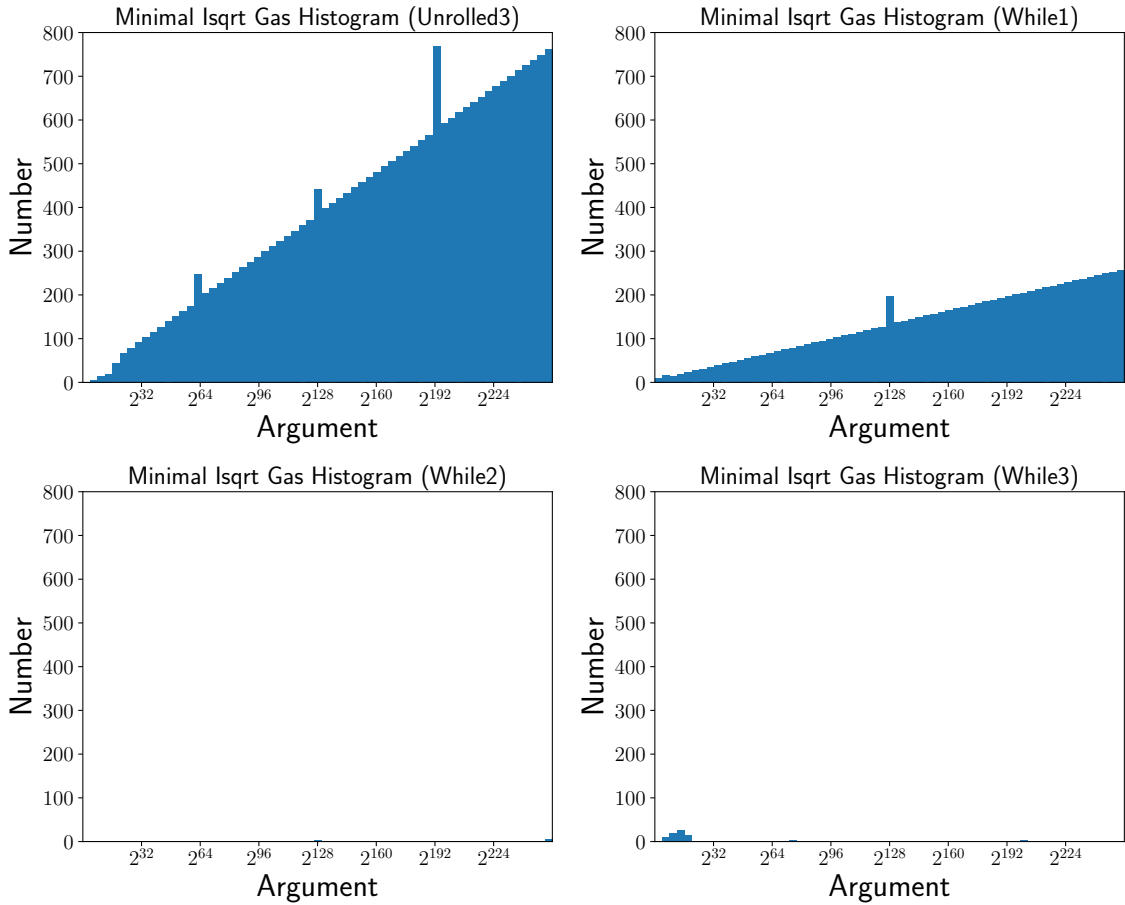


Figure 2: Here we plot a histogram showing where each algorithm is minimal when using a larger sample of deterministic values. Each bin counts the total number of instances where the algorithm’s gas cost is minimal. These results are for the tests in Appendix A.1.

Total	16 384
Unrolled3	15 774
While1	190
While2	117
While3	303

Table 7: Here are the number of times each method had minimal gas cost; these are results for the additional loguniform random values. These results are for the tests in Appendix A.2.

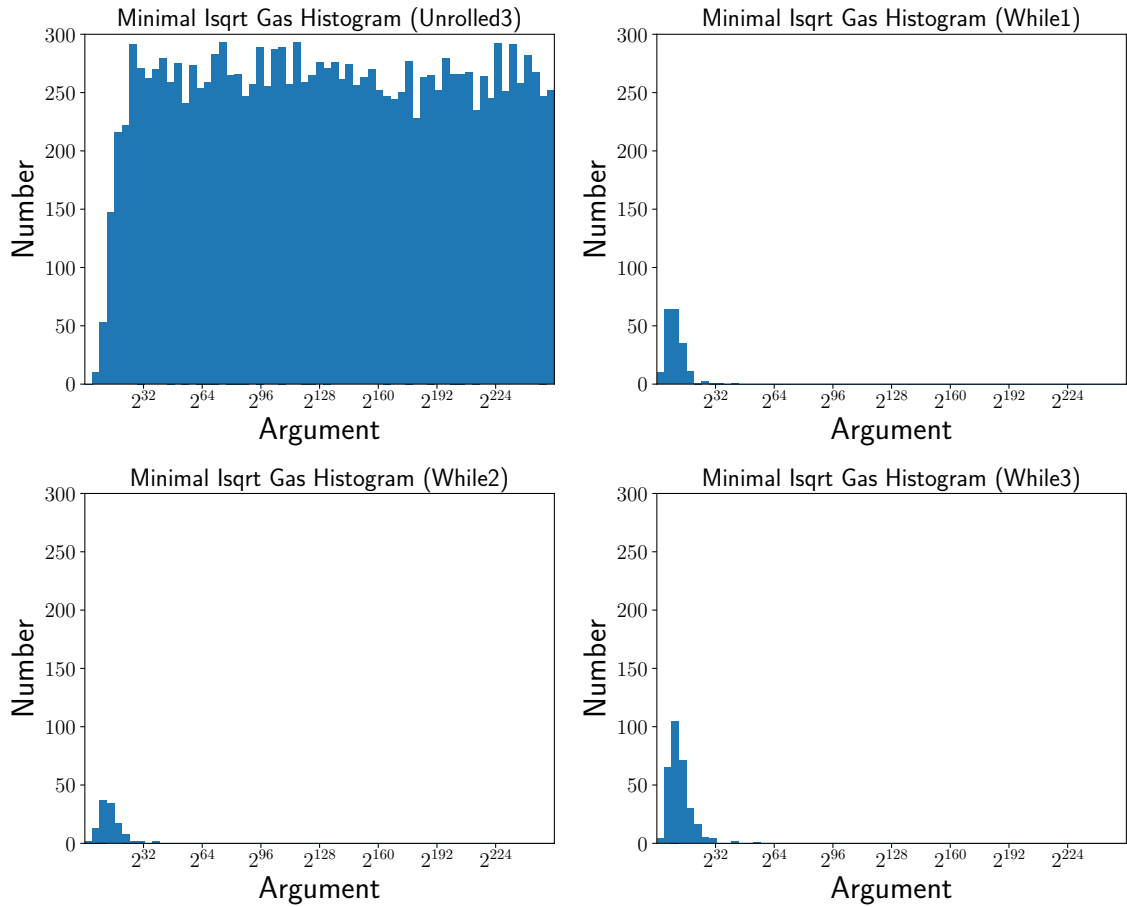


Figure 3: Here we plot a histogram showing where each algorithm is minimal when using a larger sample of loguniform random values. Each bin counts the total number of instances where the algorithm’s gas cost is minimal. These results are for the tests in Appendix A.2.

B.1 Error Estimates Based on Newton's Method

From [28, Chapter 3, Exercise 16], we recall that Newton's method for computing square roots has the form

$$x_{k+1} := \frac{1}{2} \left(x_k + \frac{\alpha}{x_k} \right) \quad (\text{B.1})$$

and that if

$$\begin{aligned} f_\alpha(t) &:= \frac{1}{2} \left(t + \frac{\alpha}{t} \right) \\ \varepsilon &:= t - \sqrt{\alpha} \end{aligned} \quad (\text{B.2})$$

then we have

$$f_\alpha(t) - \sqrt{\alpha} = \frac{\varepsilon^2}{2t}. \quad (\text{B.3})$$

We are particularly interested in asymptotic error bounds in order to determine the maximum required number of Newton iterations. To do this, we assume

$$2^{e-1} \leq \sqrt{\alpha} < 2^e \quad (\text{B.4})$$

and

$$|\varepsilon_0| \leq 2^{e-b_0}. \quad (\text{B.5})$$

This initial error bound allows us to compute

$$|\varepsilon_1| \leq 2^{e-2b_0}. \quad (\text{B.6})$$

Iterating the error bound, we see that

$$|\varepsilon_k| \leq 2^{e-2^k b_0}. \quad (\text{B.7})$$

The error decay rate in Eq. (B.7) is called *quadratic convergence*: the number of correct digits approximately *doubles* every iteration.

We want to find k such that

$$|\varepsilon_k| \leq 1, \quad (\text{B.8})$$

and this is ensured when

$$2^{e-2^k b_0} \leq 1. \quad (\text{B.9})$$

For this value of k , we have

$$1 < 2^{e-2^{k-1} b_0}. \quad (\text{B.10})$$

Rearranging, we find

$$k < \lg e - \lg b_0 + 1 \quad (\text{B.11})$$

and can be approximated as

$$k \simeq \lg \lg \sqrt{\alpha}. \quad (\text{B.12})$$

B.2 Error Estimates Based on Halley's Method

The quadratic convergence (doubling of correct digits approximately every iteration) makes Newton's method very popular. This lead to the development of higher-order methods. In particular, Halley's method has *cubic convergence*: the number of correct digits approximately *triples* every iteration. This faster convergence comes at the cost of more expensive iterations.

Halley's method for computing square roots [15, Section 2] has the form

$$x_{k+1} = \frac{x_k^3 + 3\alpha x_k}{3x_k^2 + \alpha} \quad (\text{B.13})$$

and that if

$$\begin{aligned} h_\alpha(t) &:= \frac{t^3 + 3\alpha t}{3t^2 + \alpha} \\ \varepsilon &:= t - \sqrt{\alpha}, \end{aligned} \quad (\text{B.14})$$

then we have

$$h_\alpha(t) - \sqrt{\alpha} = \frac{\varepsilon^3}{3t^2 + \alpha}. \quad (\text{B.15})$$

Follow the same procedure from Appendix B.1, we find k such that

$$0 \leq \varepsilon_k \leq 2^{e-3^k b_0} \leq 1. \quad (\text{B.16})$$

This leads to

$$1 < 2^{e-3^{k-1} b_0} \quad (\text{B.17})$$

and reduces to

$$k < \frac{1}{\lg 3} \lg e - \frac{1}{\lg 3} \lg b_0 + 1. \quad (\text{B.18})$$

After noting $(\lg 3)^{-1} \simeq 0.63$, we see

$$k \simeq 0.63 \lg \lg \sqrt{\alpha}. \quad (\text{B.19})$$

This leads to an asymptotic reduction in the required number of iteration steps at a greater per-iteration cost.

While Halley's method is better asymptotically, we see in Appendix D.4 that this method is not competitive against Newton's method for the values of interest (uint256).

B.3 Higher-order Iterative Methods

Higher-order generalizations are possible, but we do not pursue them here. That Halley’s method is not competitive with Newton’s method suggests that better *practical* algorithms will not come from faster *asymptotic* methods. Furthermore, we note that deriving Newton’s method for square roots involves computing the first derivative of $f(t) = t^2 - \alpha$ while Halley’s method requires computing $f''(t)$. Given that $f^{(\ell)}(t) = 0$ for $\ell \geq 3$, the author expects higher order methods will not converge faster than Halley.

C Gas Costs

We look at the specific gas costs of some operations; these will be useful in later discussions when making comparisons between different methods.

C.1 Initializations

We look at initialization costs associated with the algorithms; see Table 8 for the results. All of these results are from initializations which are written in assembly (Yul [32]).

Most initializations are self-explanatory; note that `Newton1` is used by `Unrolled1` and `While1`, `Newton2` is used by `Unrolled2` and `While2`⁷, and `Newton3` is used by `Unrolled3` and `While3`. The test used the value $2^{256} - 1$ to compute the gas cost. We ignore return logic but include gas costs associated with getting the input to and from the proper range; this is done for `Linear`, `Hyper4`, `Lookup4`, and `Lookup8`. This separates the initialization and return logic costs from the Newton iteration costs.

`Linear` has the lowest initialization out of the non-`Newtons`; this shows why the method had good results in Table 4. Similarly, `Hyper4` and `Lookup4` have comparable initialization costs and are competitive with `Linear`. `Lookup8` has an expensive initialization; this suggests other lookup-table-based methods will not be competitive.

These results would seem to imply that more complex initializations will not lead to overall improvements. The inclusion of `clz` as an opcode makes determining the bitlength relatively cheap (and unlikely to be improved further); this means that the trade off is primarily between low-cost initializations and the required Newton iterations.

C.2 Iterations

C.2.1 Newton Iteration

The data in Table 9 shows each Newton iteration costs 20 gas⁸. These tests used the value $3 \cdot 2^{160}$ (to enable a direct comparison with the Halley iteration results in Appendix C.2.3); the `Newton3` initialization is used, although there is no return logic.

⁷It is unclear to the author why `Newton1` has a gas cost of 106 while `Newton2` has a gas cost of 117; he expects they should be equal as `Unrolled1` and `Unrolled2` have the same gas cost of 275 in Table 3.

⁸The unrolled Newton cost was previously determined to be 48 gas per iteration for a *non-assembly* implementation. Thus, switching to assembly provides significant gas savings.

Method	Gas
Newton1	106
Newton2	117
Newton3	138
BitLength	192
Linear	153
Hyper4	161
Lookup4	162
Lookup8	322

Table 8: Here we give the gas cost for initialization methods; note that we ignored the initial check and did not include return logic. `Newton1` is used by `Unrolled1` and `While1`; `Newton2` is used by `Unrolled2` and `While2`; and `Newton3` is used by `Unrolled3` and `While3`. The value $2^{256} - 1$ was used. This clearly shows that some of the more complex initialization methods cost more gas.

Newton Iterations	Gas
0	138
1	158
2	178
3	198
4	218
5	238
6	258

Table 9: Here we give the gas cost for computing each Newton iteration when writing in assembly; the initialization is `Newton3`. The difference between k and $k + 1$ iterations is 20 gas. The value $3 \cdot 2^{160}$ was used.

Value	While Iterations	Gas
$2^{256} - 2^{128}$	1	275
$2^{256} - 2^{224}$	2	340
$2^{256} - 2^{240}$	3	405
$2^{256} - 2^{248}$	4	470
$2^{256} - 2^{252}$	5	535
$2^{256} - 2^{254}$	6	600

Table 10: Here we give the gas cost for computing each `While2` written in assembly, which allows us to determine the cost per `while` loop. The difference between k and $k + 1$ iterations is 65 gas.

Halley Iterations	Gas
0	138
1	187
2	258
3	311
4	364

Table 11: Here we give the gas cost for computing each Halley iteration. As we can see, the difference between k and $k + 1$ iterations would seem to be 53 gas (notwithstanding the difference between 0 and 1, which is 49, and the difference between 1 and 2, which is 71). The value $3 \cdot 2^{160}$ was used.

Value	Binary Iterations	Total Gas	Total Diff
2^3	2	416	
2^5	3	534	118
2^7	4	652	118
2^9	5	760	108
2^{13}	6	878	118
2^{15}	7	986	108

Table 12: Here we give the gas cost for computing each iteration within Binary Search. These results give an approximate iteration cost of 115 gas.

Value	Interp Iterations	Gas
$2^{256} - 2^{128}$	1	450
$2^{256} - 2^{180}$	2	624
$2^{256} - 2^{196}$	3	798
$2^{256} - 2^{224}$	4	972
$2^{256} - 2^{230}$	5	1146
$2^{256} - 2^{236}$	6	1320

Table 13: Here we give the gas cost for computing each iteration within Interpolation Search, which allows us to determine the cost per `while` loop. The difference between k and $k + 1$ iterations is 174 gas.

C.2.2 While Iteration

We choose values of the same bitlength to ensure the initialization cost is constant to determine the cost per `while` loop within `While1`, `While2`, and `While3`⁹; the required number of iterations is determined manually. The results in Table 10 show each `while` loop costs 65 gas; this makes each iteration more than triple the cost of the unrolled version¹⁰.

From this we see that while it is clear that *certain* integer square root values may be computed more efficiently using `while` loops, in general it is a more expensive operation.

C.2.3 Halley Iteration

The data in Table 11 shows each Halley iteration costs 53 gas¹¹ when written in assembly¹². These tests used the value $3 \cdot 2^{160}$ (so there is no overflow during the computation); the `Newton3` initialization is used, although there is no return logic.

Thus, even though fewer Halley iterations are required to get to the same level of accuracy, each Halley iteration is almost three times more expensive than one Newton iteration. As expected, in Appendix D.4 we see using Halley’s method is not competitive.

C.2.4 Binary Search Iteration

The data in Table 12, shows that each binary iteration costs approximately 115 gas (search algorithms will be investigated in Appendix D.5). This is more expensive than a Newton iteration within a `while` loop.

C.2.5 Interpolation Search Iteration

The results in Table 13, show that each iteration within Interpolation search costs 174 gas; this method is not competitive¹³.

D Additional Algorithms and Further Optimizations

We now investigate methods for potential improvements. We start by looking at improvements in initialization before turning to general methods. Table 14 contains the results of other algorithms performing the same standard tests from Section 6; however, none of these algorithms are competitive.

⁹This was necessary *before* `clz` was added; while it is not needed now, we continue to use the same values.

¹⁰The `while` loop cost was previously determined to be 90 gas per iteration for a *non-assembly* implementation. Thus, switching to assembly resulted in a lower gas cost but less than the unrolled case.

¹¹As noted in the table, 53 gas holds for the difference between 2 and 3 as well as 3 and 4; between 0 and 1 the difference was 49 while between 1 and 2 the difference was 71. The reason for why different iterations give different values is unclear.

¹²Previously, the *non-assembly* implementation had 135 gas cost per Halley iteration (aside from 138 gas between 0 and 1).

¹³The author admits he did not attempt to optimize the initial portion of the algorithm in Listing 22; he hopes it is clear this would be a waste of time.

	Python	Binary Search	Interpolation Search	Hardware
Max	411	15 284	13 152	14 222
Mean	411	7433	3844	11 614
Median	411	7324	3234	11 576
Std	0	4173	3054	654

Table 14: Here are statistics related to the gas cost data. All of these algorithms are provably correct, but only `Python` is close to being competitive. These results are for the tests in Section 6.

D.1 Additional Initializations

Ceteris paribus, a more accurate initialization will require fewer Newton iterations than a less accurate initialization, so we begin by discussing initialization improvements. We note, though, the top algorithms (`Unrolled3`, `Unrolled1`, `Unrolled2`) only use one or two bits of initial precision.

D.1.1 Polynomial Approximations

The results of `Linear` in Table 4 show that a linear (affine) approximation provides essentially 4 correct initial bits. It is possible to extend the approximation to higher degree polynomials; however, the results in Appendix C.1 suggest the gas required to get a more accurate initialization will cost more than the savings in reduced Newton iterations (20 gas per iteration as noted in Appendix C.2.1).

D.1.2 Rational Approximations

The results `Hyper4` in Table 4 shows that a hyperbola provides a good initial approximation; however, `Hyper4` is not more efficient than `Unrolled3` or `Linear`. The discussion in Appendix D.1.1 is worth reiterating: Newton iterations are cheap and good approximations are expensive. It seems unlikely that a rational function which gives 8 bits of accuracy will result in a lower overall gas cost¹⁴.

D.1.3 Lookup Tables

The results in Table 4 show `Lookup4` and `Lookup8` produce decent results¹⁵; however, even though `Lookup8` performs one less Newton iteration than `Lookup4` it costs significantly more gas. This leads us to expect that higher precision lookup tables (which would be “`Lookup16`”) will result in higher gas costs (notwithstanding the cost of *storing* all the approximations,

¹⁴The author spent a substantial amount of time *finding* the 4-bit approximations used in `Linear` and `Hyper4`; he was unable to find 5-bit approximations. The poor performance of `BitLength` (see Table 4) strongly suggests that no piecewise approximation will be competitive. This would appear to suggest the best case would be to find a hyperbolic approximation (with the same form as `Hyper4`) with 8 bits of initial accuracy. The author thinks an approximation of 5 bits could be found; he does not think 8 bits is possible.

¹⁵We note that implementing `Lookup4` in assembly resulted in an improvement in ranking to the fifth best; the author was surprised to see this.

which would require a *much larger* deployment cost). Furthermore, in Table 8 we see `Lookup8` has the most expensive initialization cost¹⁶.

D.2 Integer Square Roots via Inverse Square Roots

In [19], the method for computing integer square roots is actually based around computing *inverse square roots*. We note, though, that the setting and constraints are different as [19] focuses on the inverse square root algorithm in order to not have to perform division. As noted in Table 1, within the Ethereum Virtual Machine the cost of integer multiplication and division is the same. Thus, although it would be an interesting exercise to implement the algorithm in Solidity, the author does not expect it to produce an efficient algorithm (that is, better than `Unrolled3`).

D.3 Approximate Square Roots

One method based on *approximate square roots* may be found in [8]. The algorithm is designated as `Python` (Listing 18); this algorithm was previously presented in [13] (though the assembly implementation presented here is new). The algorithm is provably correct but is not as efficient as the other Newton-based methods; see the results in Table 14.

D.4 Halley’s Method and Higher-Order Approximations

We have primarily focused on variations of Newton’s method. The asymptotic analysis in Appendix B showed that fewer Halley iterations are required in comparison to Newton iterations. Unfortunately, in Appendix C.2 we saw that Halley iterations are almost three times more expensive than Newton iterations (notwithstanding issues around overflow).

For completeness, we include a version of Halley’s method in Listing 19. This method is of limited value as the code will overflow when the input is too large, and we have not proven the return logic is valid. The initialization is the same as `Unrolled3`, and this is required in order to only have to perform 4 Halley iterations; using the initializations of `Unrolled1` or `Unrolled2` would require 5 Halley iterations. See Appendix E.3.9 for the proof that 4 iterations is sufficient.

We include results of running Halley’s method on $3 \cdot 2^{160}$ in Table 15; this is why the author does not expect higher-order approximations to produce practical improvements.

D.5 Search Algorithms

We turn our attention to search algorithms. The idea of using search algorithms comes from [35, Chapter 11-1 Integer Square Root, Binary Search]. At the end of the analysis, we will see that these methods are not competitive; this can be seen from the results in Table 14.

The noncompetitive nature appears to come from the inherent complexity of search algorithms: in [18, Page 425, Exercise 6.2.1.22], Knuth mentions that on average $\lg \lg N$

¹⁶While the author does not claim to have produced the most efficient implementation of `Lookup8` (or any other algorithm for that matter), he is unable to see how it could be reduced anywhere close to the gas cost of `Lookup4`.

Listing 18: Provably correct method for computing Integer Square Roots based on [8]; this algorithm was previously presented in [13].

```
// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 result) {
    assembly ("memory-safe") {
        let e := clz(x)
        e := shr(1, e)
        let m := shl(shl(1, e), x)

        result := add(1, shr(254, m))
        result := add(shl(1, result), div(shr(251, m), result))
        result := add(shl(3, result), div(shr(245, m), result))
        result := add(shl(7, result), div(shr(233, m), result))
        result := add(shl(15, result), div(shr(209, m), result))
        result := add(shl(31, result), div(shr(161, m), result))
        result := add(shl(63, result), div(shr(65, m), result))
        result := shr(e, result)

        result := sub(result, gt(result, div(x, result)))
    }
}
```

Method	Gas
Halley's Method	387
Unrolled3	269

Table 15: Here we compare running Halley's method from Listing 19 with Unrolled3 (Listing 9); the value $3 \cdot 2^{160}$ was used. Even if it were possible to remove one Halley iteration, the total gas would only decrease from 387 to 334 (which is still more expensive).

Listing 19: General form for computing Integer Square Roots based on Halley’s method; the return logic of this method has not been verified and the values will overflow for large inputs.

```
// SPDX-License-Identifier: 0BSD
// NOTE: return logic has not been verified.
function sqrt(uint256 x) internal pure returns (uint256 result) {
    assembly ("memory-safe") {
        // Here, result stores the "bit length" of x
        result := clz(x)
        result := sub(255, result)
        result := shr(1, shl(shr(1, result), 3))

        let r2 := mul(result, result)
        result := div(mul(result, add(r2, mul(3, x))),
                     add(mul(3, r2), x))
        r2 := mul(result, result)
        result := div(mul(result, add(r2, mul(3, x))),
                     add(mul(3, r2), x))
        r2 := mul(result, result)
        result := div(mul(result, add(r2, mul(3, x))),
                     add(mul(3, r2), x))
        r2 := mul(result, result)
        result := div(mul(result, add(r2, mul(3, x))),
                     add(mul(3, r2), x))

        // TODO: determine correct return logic
        result := sub(result, gt(result, div(x, result)))
    }
}
```

Listing 20: Binary search method for computing Integer Square Roots in C from [35]

```

int isqrt3(unsigned x) {
    unsigned a, b, m;                                // Limits and midpoint.

    a = 1;
    b = (x >> 5) + 8;                                // See text.
    if (b > 65535) b = 65535;
    do {
        m = (a + b) >> 1;
        if (m*m > x) b = m - 1;
        else          a = m + 1;
    } while (b >= a);
    return a - 1;
}

```

comparisons are required for interpolation sort and that “*all* search algorithms ... must make asymptotically $\lg \lg N$ comparisons, on the average” (emphasis in original). The required $\lg \lg N$ comparisons would appear to translate into $\lg \lg N$ loop iterations, and each loop iteration will presumably be more complex than a Newton iteration within the EVM; in Appendix C.2.4, we see the per-cost of each loop within binary search is approximately 115 gas (significantly more than the cost of a Newton loop). This would appear to rule out all generic search algorithms (even if better search algorithms only required $\lg \lg N$ iterations).

The paper referenced for the required $\lg \lg N$ comparisons on average is presumably [38], which the author of the present work is unable to access.

D.5.1 Binary Search

The idea of using *binary search* to compute the integer square root comes from its inclusion in [35]; see Listing 20 for the C code from [36] and Listing 21 for the Solidity code. We use powers-of-2 for the left and right limits to minimize the loop iterations.

By bounding the square root and then choosing the midpoint, the error is halved at every iteration. This leads to requiring $\Theta(\lg(x))$ number of iterations; in particular, at most $\frac{1}{2} \lg(x)$ iterations. Unfortunately, this means that 128 loop iterations are required for the largest integer square root values we will compute, and this leads to the large associated gas costs. Thus, this method is not competitive and no optimization will make it competitive.

D.5.2 Interpolation Search

The $\Theta(\lg(x))$ number of iterations required by binary search significantly increases the gas cost. *Interpolation search* [11] seeks to reduce the required number of iterations by using a linear approximation to update where to search next. It is possible to show the required iterations is $\Theta(\lg \lg(x))$, but this *assumes a uniform search space* (which is not the case here). The results in Table 13 show the per-iteration cost of 174 gas makes this not a viable choice.

Listing 21: Provably correct method based on binary search

```
// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 left) {
    assembly ("memory-safe") {
        // Here, result stores the "bit length" of x
        left := clz(x)
        left := sub(255, left)

        left := shl(shr(1, left), 1)
        let right := shl(1, left)

        // iszero(gt(left, right)) is equivalence to
        // left <= right
        for { } iszero(gt(left, right)) { } {
            let midpoint := shr(1, add(left, right))
            switch gt(midpoint, div(x, midpoint))
            case 1 {
                right := sub(midpoint, 1)
            }
            default {
                left := add(midpoint, 1)
            }
        }
        left := sub(left, 1)
    }
}
```

Algorithm 2 Generic Interpolation Search from [11, Page 41] (modified)

Require: `array` is a sorted array of values; $\varphi(\alpha, n)$ is the probe position to inspect.

```
1: procedure INTERPOLATIONSEARCH( $\alpha$ )
2:   low := 0
3:   high :=  $n + 1$ 
4:   while high - low > 1 do
5:      $j := \varphi((\alpha - \text{array}[\text{low}])/(\text{array}[\text{high}] - \text{array}[\text{low}]), \text{high} - \text{low} - 1) + \text{low}$ 
6:     if  $\alpha = \text{array}[j]$  then
7:       return  $j$ 
8:     else if  $\alpha < \text{array}[j]$  then
9:       high :=  $j$ 
10:    else
11:      low :=  $j$ 
12:    end if
13:  end while
14:  return fail
15: end procedure
```

The Solidity code may be found in Listing 22¹⁷.

The general idea of interpolation search makes sense: search an ordered array more quickly by estimating how the values in the array change with a linear approximation. The problem is that we are not searching a generic array but looking to compute a square root, and we can compute the exact derivative (so no approximation is needed). An analysis of interpolation search may be found in [11], and Algorithm 2 is based on the algorithm from [11, Page 41].

While more complicated search algorithms possible (like the *interpolated binary search* algorithm [22] which seeks to combine the best of both binary and interpolation search), the results here suggest that *no* search method will be competitive against Newton’s method.

D.6 Hardware Algorithm

We include an additional algorithm based on one from [35, Page 285]; see Listing 23 for the C code from [36] and Listing 24 for the Solidity code. This is a “hardware” algorithm, and the only operations used are comparisons, bitwise OR, bit shifts, and subtraction. The 128 loop iterations, though, make this not competitive; see the results in Table 14.

D.7 Potential Optimizations

Based on the cost of Newton iterations (20 gas per unrolled iteration and 53 gas within a **while** loop; see Appendix C.2), reducing the number of Newton iterations only reduces the overall gas cost so much.

¹⁷Any reader complaining about the author failing to optimize the initial portion of Listing 22 is more than welcome to do so, but the author does not care: the algorithm is nowhere close to competitive.

Listing 22: Method based on interpolation search

```
// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 left) {
    unchecked {
        if (x == 0) {
            return 0;
        }
        if (x < 16) {
            if (x < 4) {
                return 1;
            } else if (x < 9) {
                return 2;
            } else {
                return 3;
            }
        }
    }
    assembly ("memory-safe") {
        left := clz(x)
        left := sub(255, left)
        left := shl(shr(1, left), 1)
        let right := shl(1, left)

        for { } and(iszero(gt(left, div(x, left))),
                    lt(div(x, right), right)) { } {
            let interp := add(left,
                               div(sub(x, mul(left, left)),
                                   add(right, left)))

            switch lt(div(x, interp), interp)
            case 1 {
                right := interp
            }
            default {
                left := add(interp, 1)
            }
        }
        left := sub(left, 1)
    }
}
```

Listing 23: “Hardware” method for computing Integer Square Roots in C from [35]

```

int isqrt4(unsigned x) {
    unsigned m, y, b;

    m = 0x40000000;
    y = 0;
    while(m != 0) {                                // Do 16 times.
        b = y | m;
        y = y >> 1;
        if (x >= b) {
            x = x - b;
            y = y | m;
        }
        m = m >> 2;
    }
    return y;
}

```

Listing 24: Provably correct method based on hardware-efficient operations

```

// SPDX-License-Identifier: 0BSD
function sqrt(uint256 x) internal pure returns (uint256 y) {
    assembly ("memory-safe") {
        // m == 2^254
        let m := shl(254, 1)

        // while (m != 0)
        for { } iszero(iszero(m)) { } {
            let b := or(y, m)
            y := shr(1, y)
            // if (x >= b)
            if iszero(lt(x, b)) {
                x := sub(x, b)
                y := or(y, m)
            }
            m := shr(2, m)
        }
    }
}

```

Additional Approximations As mentioned in Appendix D.1, it is expected that polynomial estimates, rational approximations, and lookup tables will not lead to an overall reduction in gas. It is unclear where additional approximation methods would come from. The best algorithms presented here (`Unrolled3` (Listing 9) followed closely by `Unrolled1` (Listing 7), `Unrolled2` (Listing 8), and `Linear` (Listing 14)) produce 2, 1, or 4 bits of initial precision.

Additional Algorithms We implemented algorithms based on Newton’s method and its generalizations, search algorithms, and approximate square roots. It is unclear to the author what other methods could lead to a competitive algorithm. From the algorithms investigated here, the method would essentially require quadratic convergence.

Assembly and Ethereum Bytecode All new algorithms were written in assembly to reduce gas costs. This improved performance and the relative ranking mostly stayed the same; notable exceptions are `Unrolled1` and `Unrolled2` jumping to second and `Lookup4` rising to fifth. While the author does not claim the implementations are optimal, he sees no obvious improvements (as the algorithms are relatively straightforward and the edge case of 0 requires no special logic). This leaves further improvements to viewing the Ethereum bytecode generated by the compiler and determining improvements from there. We leave this task to the interested reader.

E Proofs

Here we include proofs for the new algorithms. This includes some of the material from [13, Appendix B]. A notable omission is the proof of `Python` (Listing 18); this is due to the fact that the emphasis here is on Newton-based algorithms. A proof of correctness for `Python` may be found in [13, Appendix C]. Some of the work here is similar to that found in [13, Appendix A] but is included for this document to be self-contained.

E.1 Mathematical Review

E.1.1 Newton Iteration over the Real Numbers

Fix $\alpha > 0$. For any $t > 0$, we define

$$f_\alpha(t) := \frac{1}{2} \left(t + \frac{\alpha}{t} \right). \quad (\text{E.1})$$

We always have

$$f_\alpha(t) = \frac{1}{2} \left(t + \frac{\alpha}{t} \right) \geq \sqrt{\alpha}. \quad (\text{E.2})$$

This follows from the inequality of the arithmetic and geometric means (AM-GM inequality); one reference is [30, Page 20]. Given $t > 0$, we can look at the sequence $\{t_k\}_{k=0}^\infty$ defined by

$$t_k = \begin{cases} t & k = 0 \\ f_\alpha(t_{k-1}) & k > 0 \end{cases}. \quad (\text{E.3})$$

Each application of $f_\alpha(\cdot)$ is called a *Newton iteration*. This sequence converges to $\sqrt{\alpha}$.

In order to see how the error changes with each iteration, we set

$$\varepsilon := t - \sqrt{\alpha}. \quad (\text{E.4})$$

From here, we find

$$f_\alpha(t) - \sqrt{\alpha} = \frac{\varepsilon^2}{2t}. \quad (\text{E.5})$$

The “ ε^2 ” in the error term shows what we mean when we say the algorithm has quadratic convergence.

We make another important observation: when $t \geq \sqrt{\alpha}$, we have

$$\begin{aligned} f_\alpha(t) - \sqrt{\alpha} &= \frac{\varepsilon^2}{2t} \\ &\leq \frac{\varepsilon^2}{2\sqrt{\alpha}}. \end{aligned} \quad (\text{E.6})$$

This fact allows us to simplify the error bounds we compute. We note that all values of the sequence in Eq. (E.3) satisfy $t_k \geq \sqrt{\alpha}$ with the possible exception of t_0 .

E.1.2 Newton Iteration over the Integers

The fact that we care about *integer operations* within the Ethereum Virtual Machine means we focus on a slightly different function. Fix $n \in \mathbb{N}$ and let $x \in \mathbb{N}$. We define

$$g_n(x) := \left\lfloor \frac{1}{2} \left(x + \frac{n}{x} \right) \right\rfloor. \quad (\text{E.7})$$

This represents the integer version of f_α .

E.2 Initializations

Throughout we assume

$$2^{f-1} \leq n < 2^f, \quad n \neq 0. \quad (\text{E.8})$$

Every algorithm has been verified to return the correct result when $n = 0$.

E.2.1 Unrolled1 and While1

We know

$$\begin{aligned} t &= 255 - \text{clz}(n) \\ &= f - 1. \end{aligned} \tag{E.9}$$

We see

$$r = 2^{\lfloor t/2 \rfloor} = 2^{e-1}. \tag{E.10}$$

when $f = 2e - k$, $k \in \{0, 1\}$.

E.2.2 Unrolled2 and While2

We know

$$\begin{aligned} t &= 257 - \text{clz}(n) \\ &= f + 1. \end{aligned} \tag{E.11}$$

We see

$$r = 2^{\lfloor t/2 \rfloor} = 2^e. \tag{E.12}$$

when $f = 2e - k$, $k \in \{0, 1\}$.

E.2.3 Unrolled3 and While3

We know

$$\begin{aligned} t &= 255 - \text{clz}(n) \\ &= f - 1. \end{aligned} \tag{E.13}$$

We see

$$r = \left\lfloor \frac{3 \cdot 2^{\lfloor t/2 \rfloor}}{2} \right\rfloor = 2^{e-1} + 2^{e-2} \tag{E.14}$$

when $f = 2e - k$, $k \in \{0, 1\}$ for $e \geq 2$. When $e = 1$, we see

$$r = \left\lfloor \frac{3}{2} \right\rfloor = 1. \tag{E.15}$$

E.2.4 BitLength

We know

$$\begin{aligned} t &= 255 - \text{clz}(n) \\ &= f - 1. \end{aligned} \tag{E.16}$$

Thus, the initialization gives the bitlength minus one.

E.2.5 Linear, Hyper4, Lookup4, and Lookup8

We know

$$t = \text{clz}(n) \tag{E.17}$$

and so $t + f = 256$. We suppose $t = 2\ell + j$ for $j \in \{0, 1\}$. If $s = \lfloor t/2 \rfloor$, then we let

$$m := 2^{2s}n. \tag{E.18}$$

We see

$$\begin{aligned} m &= 2^{2s}n \\ &< 2^{2\ell+f} \\ &\leq 2^{256}. \end{aligned} \tag{E.19}$$

We also have

$$\begin{aligned} m &= 2^{2s}n \\ &\geq 2^{2\ell+f-1} \\ &\geq 2^{254}. \end{aligned} \tag{E.20}$$

Together, these imply

$$2^{254} \leq m < 2^{256}, \tag{E.21}$$

as desired. This scaling allows for the specific algorithms to properly handle the error bound computations below.

E.3 Error Bounds

Based on the above work, we will now compute the error bounds for a number of algorithms. The computations are similar, and we primarily focus on the initial error and the first Newton iteration, as these are most important. Only when care is required to prove the desired error bounds do we go into greater detail. Throughout we assume $n \geq 1$.

Throughout this section, we are assuming that $\sqrt{n}$ is an e -bit number:

$$2^{e-1} \leq \sqrt{n} < 2^e. \quad (\text{E.22})$$

This allows us to state the initialization value; different initialization *algorithms* may arrive at the same initialization *value*.

Throughout, we will specify the initialization x_0 and then look at

$$x_k = g_n(x_{k-1}), \quad k > 0. \quad (\text{E.23})$$

We are interested in the values

$$\varepsilon_k := x_k - \sqrt{n}. \quad (\text{E.24})$$

These values will allow us to determine when x_k has reached $\text{ISQRT}(n)$.

Technically, we care about these error values:

$$\hat{\varepsilon}_k := x_k - \lfloor \sqrt{n} \rfloor. \quad (\text{E.25})$$

This is because we are computing *integer square roots*, not square roots. At the same time, we note that when $\varepsilon_k \leq 1$,

$$x_k \in \{q, q+1\}, \quad q = \lfloor \sqrt{n} \rfloor. \quad (\text{E.26})$$

This follows because $x_k \in \mathbb{N}$, $x_k \geq q$, and $|\sqrt{n} - \lfloor \sqrt{n} \rfloor| \leq 1$. Thus, we focus on determining when $\varepsilon_k \leq 1$. Any attempt to get a smaller error bound is pointless because it is less expensive to check if x_k satisfies $x_k^2 \leq n$.

In each error bound calculation, we will assume that $x_k > \sqrt{n}$; if this ever fails, we know that $x_k = \lfloor \sqrt{n} \rfloor$.

E.3.1 Unrolled1

The initialization value is chosen to be the largest power-of-two less than or equal to $\sqrt{n}$:

$$x_0 = 2^{e-1}. \quad (\text{E.27})$$

We have the error bound

$$|\varepsilon_0| \leq 2^{e-1}. \quad (\text{E.28})$$

We now use this to bound ε_1 :

$$\begin{aligned} \varepsilon_1 &= \frac{\varepsilon_0^2}{2x_0} \\ &\leq 2^{e-2}. \end{aligned} \quad (\text{E.29})$$

Continuing, we find

$$\begin{aligned}
\varepsilon_2 &\leq 2^{e-4} \\
\varepsilon_3 &\leq 2^{e-8} \\
&\vdots
\end{aligned} \tag{E.30}$$

E.3.2 Unrolled2

The initialization value is chosen to be the smallest power-of-two greater than to $\sqrt{n}$:

$$x_0 = 2^e. \tag{E.31}$$

Note the strict inequality: we have $x_0 > \sqrt{n}$. We have the error bound

$$\varepsilon_0 \leq 2^{e-1}. \tag{E.32}$$

We now use this to bound ε_1 :

$$\begin{aligned}
\varepsilon_1 &= \frac{\varepsilon_0^2}{2x_0} \\
&\leq 2^{e-3}.
\end{aligned} \tag{E.33}$$

Continuing, we find

$$\begin{aligned}
\varepsilon_2 &\leq 2^{e-6} \\
\varepsilon_3 &\leq 2^{e-12} \\
&\vdots
\end{aligned} \tag{E.34}$$

This shows that **Unrolled2** converges slightly faster (has smaller error) than **Unrolled1**. This is due solely to the larger initial starting value.

E.3.3 Unrolled3

The initialization value is chosen to be the arithmetic mean of the initializations for **Unrolled1** and **Unrolled2**:

$$x_0 = 2^{e-1} + 2^{e-2}. \tag{E.35}$$

We have the error bound

$$|\varepsilon_0| \leq 2^{e-2}. \tag{E.36}$$

We now use this to bound ε_1 :

$$\begin{aligned}\varepsilon_1 &= \frac{\varepsilon_0^2}{2x_0} \\ &\leq 2^{e-4.5}.\end{aligned}\tag{E.37}$$

Here we are using the fact that $\log_2(1/3) < -1.5$. Continuing, we find

$$\begin{aligned}\varepsilon_2 &\leq 2^{e-9} \\ \varepsilon_3 &\leq 2^{e-18} \\ &\vdots\end{aligned}\tag{E.38}$$

E.3.4 BitLength

We use the following approximation

$$x_0 = \begin{cases} \lfloor (27 \cdot 2^{e-1})/2^4 \rfloor & \text{BITLENGTH}(n) = 0 \pmod{2} \\ \lfloor (39 \cdot 2^{e-1})/2^5 \rfloor & \text{BITLENGTH}(n) = 1 \pmod{2} \end{cases}.\tag{E.39}$$

We assume n is an f -bit number:

$$2^{f-1} \leq n < 2^f, \quad f = 2e - k, \quad k \in \{0, 1\}.\tag{E.40}$$

- Case 0: BITLENGTH(n) is even or $k = 0$.

In this case, we have

$$2^{2e-1} \leq n < 2^{2e},\tag{E.41}$$

whence it follows that

$$\sqrt{2} \cdot 2^{e-1} \leq \sqrt{n} < 2^e.\tag{E.42}$$

Using the fact $45/32 < \sqrt{2}$, we have

$$45 \cdot 2^{e-6} \leq \sqrt{n} < 64 \cdot 2^{e-6}.\tag{E.43}$$

Because we are choosing

$$x_0 := 27 \cdot 2^{e-5},\tag{E.44}$$

we have

$$|\varepsilon_0| \leq 5 \cdot 2^{e-5}.\tag{E.45}$$

It follows that

$$\begin{aligned}
\varepsilon_1 &= \frac{\varepsilon_0^2}{2x_0} \\
&\leq \frac{25}{27} 2^{e-6} \\
&\leq 2^{e-6}.
\end{aligned} \tag{E.46}$$

- Case 1: $\text{BITLENGTH}(n)$ is odd or $k = 1$.

In this case, we have

$$2^{2e-2} \leq n < 2^{2e-1}, \tag{E.47}$$

whence it follows that

$$2^{e-1} \leq \sqrt{n} < \sqrt{2} \cdot 2^{e-1}. \tag{E.48}$$

Using the fact $\sqrt{2} < 23/16$, we have

$$16 \cdot 2^{e-5} \leq \sqrt{n} < 23 \cdot 2^{e-5}. \tag{E.49}$$

Because we are choosing

$$x_0 := 39 \cdot 2^{e-6}, \tag{E.50}$$

we have

$$|\varepsilon_0| \leq 7 \cdot 2^{e-6}. \tag{E.51}$$

It follows that

$$\begin{aligned}
\varepsilon_1 &= \frac{\varepsilon_0^2}{2x_0} \\
&\leq \frac{49}{39} 2^{e-7} \\
&\leq 2^{e-6.65},
\end{aligned} \tag{E.52}$$

where we are using the bound $\log_2(49/39) < 0.35$.

In the worst case, we have

$$\begin{aligned}\varepsilon_2 &\leq 2^{e-12} \\ \varepsilon_3 &\leq 2^{e-24} \\ &\vdots\end{aligned}\tag{E.53}$$

We note that it is possible to get better error bounds, but they are not worth the effort.

E.3.5 Linear

We start by scaling n (reabeled as m) so that

$$2^{2e-2} \leq m < 2^{2e}\tag{E.54}$$

for $e = 128$. We then set

$$v := \left\lfloor \frac{m}{2^{2e-4}} \right\rfloor\tag{E.55}$$

so that

$$v \cdot 2^{2e-4} \leq m < (v+1) \cdot 2^{2e-4}, \quad v \in \{4, 5, \dots, 15\}.\tag{E.56}$$

This allows us to make the reduction to

$$\sqrt{v} \cdot 2^{e-2} \leq \sqrt{m} < \sqrt{v+1} \cdot 2^{e-2}.\tag{E.57}$$

In order to determine the appropriate approximation, we first find bounds for $\sqrt{v}$ and $\sqrt{v+1}$. In particular, we computed values $\nu_{v,0}, \nu_{v,1} \in \mathbb{N}$ with

$$\begin{aligned}\frac{\nu_{v,0}}{16} &\leq \sqrt{v} \\ \frac{\nu_{v,1}}{16} &\geq \sqrt{v+1}.\end{aligned}\tag{E.58}$$

Using these values, we let

$$\mu_v := 2 \cdot (14 + v).\tag{E.59}$$

The initial approximation is

$$x_0 = \mu_v \cdot 2^{e-6}.\tag{E.60}$$

We separate error calculations into two cases:

- Case 1: $v \in \{4, 5, 6, 7, 8\}$

In this case, the initial error is

$$\varepsilon_0 \leq 2^{e-4} \quad (\text{E.61})$$

and we have the error bounds

$$\begin{aligned} \varepsilon_1 &\leq 2^{e-8} \\ \varepsilon_2 &\leq 2^{e-16} \\ &\vdots \end{aligned} \quad (\text{E.62})$$

- Case 2: $v \in \{9, 10, 11, 12, 13, 14, 15\}$

This case is more involved. We focus on the case $v = 9$. We have the bound

$$48 \cdot 2^{e-6} \leq \sqrt{m} < 51 \cdot 2^{e-6} \quad (\text{E.63})$$

with

$$\mu_9 = 46 \cdot 2^{e-6}. \quad (\text{E.64})$$

As we can see, $|\varepsilon_0| \leq 5 \cdot 2^{e-6}$. The error bound after the first Newton iteration is

$$\begin{aligned} \varepsilon_1 &= \frac{\varepsilon_0^2}{2x_0} \\ &\leq \frac{25}{23} 2^{e-8} \\ &\leq 2^{e-7.875}. \end{aligned} \quad (\text{E.65})$$

Here, we are using the bound $\log_2(25/23) < 0.125$. The second iteration bound gives

$$\begin{aligned} \varepsilon_2 &= \frac{\varepsilon_1^2}{2x_1} \\ &\leq \frac{1}{24} 2^{e-11.75} \\ &\leq 2^{e-16.25}. \end{aligned} \quad (\text{E.66})$$

Here, we are using the bound of $x_1 \geq 48 \cdot 2^{e-6} = 24 \cdot 2^{e-5}$ from Eq. (E.63); we also use the bound $\log_2(1/24) < -4.5$. This bound for x_k continues to be used for the next error bounds:

$$\begin{aligned}
\varepsilon_3 &\leq 2^{e-33} \\
\varepsilon_4 &\leq 2^{e-66.5} \\
\varepsilon_5 &\leq 2^{e-133.5} \\
&\vdots
\end{aligned} \tag{E.67}$$

Mutatis mutandis, the cases of $v \in \{10, 11, 12, 13, 14, 15\}$ follow; in these examples, we have $|\varepsilon_0| \leq 3 \cdot 2^{e-5}$. The important fact is that, in all cases, $\varepsilon_5 \leq 2^{e-128}$. We note that the case $v = 10$ is shown in Appendix [E.3.6](#).

E.3.6 Hyper4

Note that the error bound derivation here is essentially the same as that of [Linear](#) in Appendix [E.3.5](#). The analysis is the same; the difference is the initialization value.

We start by scaling n (reabeled as m) so that

$$2^{2e-2} \leq m < 2^{2e} \tag{E.68}$$

for $e = 128$. We then set

$$v := \left\lfloor \frac{m}{2^{2e-4}} \right\rfloor \tag{E.69}$$

so that

$$v \cdot 2^{2e-4} \leq m < (v+1) \cdot 2^{2e-4}, \quad v \in \{4, 5, \dots, 15\}. \tag{E.70}$$

This allows us to make the reduction to

$$\sqrt{v} \cdot 2^{e-2} \leq \sqrt{m} < \sqrt{v+1} \cdot 2^{e-2}. \tag{E.71}$$

In order to determine the appropriate approximation, we first find bounds for $\sqrt{v}$ and $\sqrt{v+1}$. In particular, we computed values $\nu_{v,0}, \nu_{v,1} \in \mathbb{N}$ with

$$\begin{aligned}
\frac{\nu_{v,0}}{16} &\leq \sqrt{v} \\
\frac{\nu_{v,1}}{16} &\geq \sqrt{v+1}.
\end{aligned} \tag{E.72}$$

Using these values, we set $\mu_v \in \mathbb{N}$ with

$$\mu_v := 2 \left\lfloor \frac{512}{31-v} \right\rfloor. \tag{E.73}$$

The initial approximation is

$$x_0 = \mu_v \cdot 2^{e-6}. \tag{E.74}$$

We separate error calculations into two cases:

- Case 1: $v \in \{4, 5, 6, 7, 8, 13, 14, 15\}$

In this case, the initial error is

$$\varepsilon_0 \leq 2^{e-4} \quad (\text{E.75})$$

and we have the error bounds

$$\begin{aligned} \varepsilon_1 &\leq 2^{e-8} \\ \varepsilon_2 &\leq 2^{e-16} \\ &\vdots \end{aligned} \quad (\text{E.76})$$

- Case 2: $v \in \{9, 10, 11, 12\}$

This case is more involved. We focus on the case $v = 10$ so as to not repeat the work from **Linear** in Appendix [E.3.5](#). We have the bound

$$25 \cdot 2^{e-5} \leq \sqrt{m} < 27 \cdot 2^{e-5} \quad (\text{E.77})$$

with

$$\mu_{10} = 24 \cdot 2^{e-5}. \quad (\text{E.78})$$

We have absorbed the common factor of 2 present in the previous two equations. As we can see, $|\varepsilon_0| \leq 3 \cdot 2^{e-5}$. The error bound after the first Newton iteration is

$$\begin{aligned} \varepsilon_1 &= \frac{\varepsilon_0^2}{2x_0} \\ &\leq \frac{9}{24} 2^{e-6} \\ &\leq 2^{e-7.4}. \end{aligned} \quad (\text{E.79})$$

Here, we are using the bound $\log_2(9/24) < -1.4$. The second iteration bound gives

$$\begin{aligned} \varepsilon_2 &= \frac{\varepsilon_1^2}{2x_1} \\ &\leq \frac{1}{25} 2^{e-10.8} \\ &\leq 2^{e-15.44}. \end{aligned} \quad (\text{E.80})$$

Here, we are using the bound of $x_1 \geq 25 \cdot 2^{e-5}$ from Eq. ([E.77](#)); we also use the bound $\log_2(1/25) < -4.64$. This bound for x_k continues to be used for the next error bounds:

$$\begin{aligned}
\varepsilon_3 &\leq 2^{e-31.52} \\
\varepsilon_4 &\leq 2^{e-63.68} \\
\varepsilon_5 &\leq 2^{e-128} \\
&\vdots
\end{aligned} \tag{E.81}$$

The case of $v = 9$ was shown in Appendix E.3.5 and the case is similar for $v \in \{11, 12\}$ as $|\varepsilon_0| \leq 3 \cdot 2^{e-5}$; we do not include the details. The important fact is that, in all cases, $\varepsilon_5 \leq 2^{e-128}$.

E.3.7 Lookup4

We start by scaling n (reabeled as m) so that

$$2^{2e-2} \leq m < 2^{2e} \tag{E.82}$$

for $e = 128$. We then set

$$v := \left\lfloor \frac{m}{2^{2e-4}} \right\rfloor \tag{E.83}$$

so that

$$v \cdot 2^{2e-4} \leq m < (v+1) \cdot 2^{2e-4}, \quad v \in \{4, 5, \dots, 15\}. \tag{E.84}$$

This allows us to make the reduction to

$$\sqrt{v} \cdot 2^{e-2} \leq \sqrt{m} < \sqrt{v+1} \cdot 2^{e-2}. \tag{E.85}$$

In order to determine the appropriate approximation, we first find bounds for $\sqrt{v}$ and $\sqrt{v+1}$. In particular, we computed values $\nu_{v,0}, \nu_{v,1} \in \mathbb{N}$ with

$$\begin{aligned}
\frac{\nu_{v,0}}{8} &\leq \sqrt{v} \\
\frac{\nu_{v,1}}{8} &\geq \sqrt{v+1}.
\end{aligned} \tag{E.86}$$

Combining Eqs. (E.85) and (E.86), we have

$$\nu_{v,0} \cdot 2^{e-5} \leq \sqrt{m} < \nu_{v,1} \cdot 2^{e-5}. \tag{E.87}$$

We can choose $\nu_{v,i}$ such that

$$\max_v |\nu_{v,1} - \nu_{v,0}| = 3. \tag{E.88}$$

This allows use to choose values $\mu_v \in \mathbb{N}$ so that

$$|\nu_{v,i} - \mu_v| \leq 2. \tag{E.89}$$

The initial approximation is

$$x_0 = \mu_v \cdot 2^{e-5} \quad (\text{E.90})$$

and the initial error is

$$\varepsilon_0 \leq 2^{e-4}. \quad (\text{E.91})$$

This gives the error bounds

$$\begin{aligned} \varepsilon_1 &\leq 2^{e-8} \\ \varepsilon_2 &\leq 2^{e-16} \\ &\vdots \end{aligned} \quad (\text{E.92})$$

E.3.8 Lookup8

We start by scaling n (relabelled as m) so that

$$2^{2e-2} \leq m < 2^{2e} \quad (\text{E.93})$$

for $e = 128$. We then set

$$v := \left\lfloor \frac{m}{2^{2e-8}} \right\rfloor \quad (\text{E.94})$$

so that

$$v \cdot 2^{2e-8} \leq m < (v+1) \cdot 2^{2e-8}, \quad v \in \{64, 65, \dots, 255\}. \quad (\text{E.95})$$

This allows us to make the reduction to

$$\sqrt{v} \cdot 2^{e-4} \leq \sqrt{m} < \sqrt{v+1} \cdot 2^{e-4}. \quad (\text{E.96})$$

In order to determine the appropriate approximation, we first find bounds for $\sqrt{v}$ and $\sqrt{v+1}$. In particular, we computed values $\nu_{v,0}, \nu_{v,1} \in \mathbb{N}$ with

$$\begin{aligned} \frac{\nu_{v,0}}{32} &\leq \sqrt{v} \\ \frac{\nu_{v,1}}{32} &\geq \sqrt{v+1}. \end{aligned} \quad (\text{E.97})$$

Combining Eqs. (E.96) and (E.97), we have

$$\nu_{v,0} \cdot 2^{e-9} \leq \sqrt{m} < \nu_{v,1} \cdot 2^{e-9}. \quad (\text{E.98})$$

We can choose $\nu_{v,i}$ such that

$$\max_v |\nu_{v,1} - \nu_{v,0}| = 3. \quad (\text{E.99})$$

This allows use to choose values $\mu_v \in \mathbb{N}$ so that

$$|\nu_{v,i} - \mu_v| \leq 2. \quad (\text{E.100})$$

The initial approximation is

$$x_0 = \mu_v \cdot 2^{e-9} \quad (\text{E.101})$$

and the initial error is

$$\varepsilon_0 \leq 2^{e-8}. \quad (\text{E.102})$$

This gives the error bounds

$$\begin{aligned} \varepsilon_1 &\leq 2^{e-16} \\ \varepsilon_2 &\leq 2^{e-32} \\ &\vdots \end{aligned} \quad (\text{E.103})$$

E.3.9 Halley Iterations

We briefly discuss the error bounds on Halley iterations required when using the **Newton3** initialization. We recall the Halley iteration error from Appendix B.2: if

$$\begin{aligned} h_\alpha(t) &:= \frac{t^3 + 3\alpha t}{3t^2 + \alpha} \\ \varepsilon &:= t - \sqrt{\alpha}, \end{aligned} \quad (\text{E.104})$$

then we have

$$h_\alpha(t) - \sqrt{\alpha} = \frac{\varepsilon^3}{3t^2 + \alpha}. \quad (\text{E.105})$$

The **Newton3** initialization gives $|\varepsilon_0| \leq 2^{e-2}$. We recall that $x_0 = 2^{e-1} + 2^{e-2}$ and compute the error bound

$$\begin{aligned} |\varepsilon_1| &= \frac{|\varepsilon_0|^3}{3x_0^2 + n} \\ &\leq \frac{2^{3e-6}}{27 \cdot 2^{2e-4} + n} \\ &\leq \frac{1}{31} 2^{e-2} \\ &\leq 2^{e-6.95}. \end{aligned} \quad (\text{E.106})$$

In the last inequality, we are using the fact $\log_2(1/31) \leq -4.95$. Continuing the same process, we compute

$$\begin{aligned}
|\varepsilon_2| &\leq 2^{e-20.85} \\
|\varepsilon_3| &\leq 2^{e-62.55} \\
|\varepsilon_4| &\leq 2^{e-187.65} \\
&\vdots
\end{aligned} \tag{E.107}$$

This shows that 4 Halley iterations is sufficient *in exact arithmetic*, but we will not perform the additional work required to ensure valid return logic for `Halley` (Listing 19).

E.4 Return Logic

We now discuss the final check performed in the new algorithms and discuss a potential issue with checks performed in other algorithms found online.

E.4.1 Possible Values at the end of Newton Iterations

In the case of algorithms performing unrolled Newton iterations, all algorithms perform iterations until $\varepsilon_k \leq 1$. This ensures that $r \in \{q, q+1\}$ with respect to error bounds. This is also ensured even if the iteration oscillates. Thus, we must ensure that we return the correct value.

E.4.2 Ensuring Proper Return Logic

We want to ensure $r^2 \leq n$. This can be done in at least two different ways.

Equivalent Forms For $n, r \in \mathbb{N}^*$ and $q := \text{ISQRT}(n)$, the following are equivalent:

1. $r \leq q$
2. $r^2 \leq n$
3. $r \leq \lfloor n/r \rfloor$.

We have 1 implies 2 as $r \leq q$ implies $r^2 \leq q^2 \leq n$. We have 2 implies 3 as $r^2 \leq n$ implies $r \leq n/r$, and $r \in \mathbb{N}$ so $r \leq \lfloor n/r \rfloor$. Finally, 3 implies 1 as $r \leq \lfloor n/r \rfloor \leq n/r$ and so $r^2 \leq n$; q is the largest integer with $q^2 \leq n$, so $r \leq q$.

Using the Equivalence Given that we have $r \in \{q, q+1\}$, we can check if $r^2 \leq n$: if this is true, then $r = q$; otherwise, $r = q+1$. While valid, this return logic may cause problems due to overflow; thus, in some of the newly developed algorithms, the case $n \geq (2^{128} - 1)^2$ is handled separately with an `if` statement at the beginning. We may also check $r \leq \lfloor n/r \rfloor$, which will not overflow (and thus is preferred).

E.4.3 Return Minimum

We note that in some of the algorithms found online (PRB (Listing 2) [2], OpenZeppelin (Listing 4) [25], and ABDK (Listing 5) [1]) the final check is of the form

$$\textbf{return} \quad \min(r, \lfloor n/r \rfloor). \quad (\text{E.108})$$

This is done after applying 7 Newton iterations.

It is not clear that this logic ensures the correct value is returned¹⁸. These algorithms have the same initialization as in `Unrolled1` and perform 7 Newton iterations, so we know $r \in \{q, q+1\}$. It is *possible* to have $r = q+1$ after 7 Newton iterations; this happens for $n = 2^{256} - 1$.

Suppose we have

$$n = q^2 + \ell, \quad \ell \in \{0, \dots, q-1\}. \quad (\text{E.109})$$

Then we see

$$n = (q-1)(q+1) + (1+\ell) \quad (\text{E.110})$$

so that

$$\left\lfloor \frac{n}{q+1} \right\rfloor = q-1. \quad (\text{E.111})$$

This shows that if n is as specified and we have $r = q+1$ after the final Newton iteration (which is possible with the current analysis), then $\min(r, \lfloor n/r \rfloor) = \lfloor n/r \rfloor = q-1$, which is incorrect. Thus, the current analysis will not ensure this return logic gives the correct result.

We note that although we have shown that it is *possible* for PRB, OpenZeppelin, and ABDK to return incorrect results (that is, results which are off by 1), we must mention that there is no known value where this error occurs. That is to say: the current analysis is *unable to prove the algorithms are correct*, but that is not the same as *proving they are incorrect*. With that said, these algorithms operate on `uint256` values and approximately half of all integers are of the form described in Eq. (E.109), so it is not possible to manually check every value to ensure a valid result is returned. Thus, if PRB, OpenZeppelin, and ABDK are to be proven correct, additional analysis is required; we leave this extended analysis to the interested reader.

E.5 Convergence Proofs

We now combine the results from the previous sections to give proofs of convergence.

¹⁸It is also unclear to the author why this was determined to be valid return logic. We note [35, Figure 11-1, Integer square root, Newton’s method] is essentially `While2` (although the initialization is modified so $\sqrt{x} \leq g_0 = 2^s$ with s minimal), so there is no concern about the return logic; the same holds for the algorithms discussed in [5, Algorithm 1.7.1], [6, Algorithm 9.2.11], and [4, Algorithm 1.13].

In looking at [8, Listing 2], the return logic is equivalent to “`if a <= (n // a); return a`”. It could be the case that returning $\min(r, \lfloor n/r \rfloor)$ was chosen as an attempt to convert the loop to a fixed iteration; this is only the author’s guess.

E.5.1 Proof of General Newton Iteration

We start by proving Algorithm 1 (from [5, Algorithm 1.7.1]) produces the correct result. This is included for completeness, and the proof provided here is essentially that of [5, Algorithm 1.7.1, Proof].

First, x decreases at every iteration. Because n is finite, the algorithm must eventually terminate. We set $q = \lfloor \sqrt{n} \rfloor$. From the discussion above, we always have $(t + n/t)/2 \geq \sqrt{n}$ for all $t > 0$, so we always have $x \geq q$. We see

$$y - x = \left\lfloor \frac{n - x^2}{2x} \right\rfloor. \quad (\text{E.112})$$

It follows that $x > \sqrt{n}$ iff $n - x^2 < 0$ iff $y < x$; therefore, $y \geq x$ implies $x \leq \sqrt{n}$. Because $x \in \mathbb{N}$ and $q \leq x \leq \sqrt{n}$, we surmise $x = q$. Thus, the algorithm returns the correct result.

Note: the only place we used the initialization value is to ensure that we initially have $x \geq \lfloor \sqrt{n} \rfloor$. Thus, provided the initialization value always satisfies this, this proof may be applied.

E.5.2 Proof of Newton Iteration Fixed Points

We now prove that for $q = \lfloor \sqrt{n} \rfloor$ we have q is a fixed point of g_n (that is, $g_n(q) = q$) iff $n + 1$ is not a square.

We let $y = g_n(x)$ as in Appendix E.5.1. The work there shows that $x > \sqrt{n}$ implies that $y < x$; additionally, for $x < q$ we have $y > x$. This holds for all n . So, we only need to investigate the case $x = q$.

From the definition of $q = \lfloor \sqrt{n} \rfloor$ we have

$$q^2 \leq n < (q + 1)^2. \quad (\text{E.113})$$

This implies

$$q^2 \leq n \leq (q + 2)q \quad (\text{E.114})$$

so that

$$\left\lfloor \frac{n}{q} \right\rfloor \in \{q, q + 1, q + 2\}. \quad (\text{E.115})$$

It follows that

$$g_n(q) = \begin{cases} q & \lfloor n/q \rfloor = q \\ q & \lfloor n/q \rfloor = q + 1 \\ q + 1 & \lfloor n/q \rfloor = q + 2 \end{cases} \quad (\text{E.116})$$

We note that when $n = a^2 - 1$ (that is, when $n + 1$ is a square) q is not a fixed point for g_n . We note that the result will oscillate as $q, q + 1, q, q + 1, \dots$

E.5.3 Unrolled1

From the initialization in Appendix E.2.1 and the error bounds in Appendix E.3.1, we see that $\varepsilon_7 \leq 1$ (we require 7 Newton iterations). Thus, $r \in \{q, q+1\}$, and the return logic discussed in Appendix E.4 ensures the correct result.

E.5.4 Unrolled2

From the initialization in Appendix E.2.2 and the error bounds in Appendix E.3.2, we see that $\varepsilon_7 \leq 1$ (we require 7 Newton iterations). Thus, $r \in \{q, q+1\}$, and the return logic discussed in Appendix E.4 ensures the correct result.

E.5.5 Unrolled3

From the initialization in Appendix E.2.3 and the error bounds in Appendix E.3.3, we see that $\varepsilon_6 \leq 1$ (we require 6 Newton iterations). Thus, $r \in \{q, q+1\}$, and the return logic discussed in Appendix E.4 ensures the correct result.

E.5.6 While1

From the initialization in Appendix E.2.1, we have $r \geq q$ after the first Newton iteration; the proof in Appendix E.5.1 then applies. The first Newton iteration is *required*.

E.5.7 While2

We use the initialization in Appendix E.2.2; the proof in Appendix E.5.1 applies because $r > \sqrt{n}$.

E.5.8 While3

From the initialization in Appendix E.2.3, we have $r \geq q$ after the first Newton iteration; the proof in Appendix E.5.1 then applies. The first Newton iteration is *required*.

E.5.9 BitLength

From the initialization in Appendix E.2.4 and the error bounds in Appendix E.3.4, we see that $\varepsilon_6 \leq 1$ (we require 6 Newton iterations). Thus, $r \in \{q, q+1\}$, and the return logic discussed in Appendix E.4 ensures the correct result.

E.5.10 Linear

Initial Scaling Appendix E.2.5 shows that we properly scale n so that $2^{254} \leq m < 2^{256}$.

Error Bounds From the error bounds in Appendix E.3.5, we see that $\varepsilon_5 \leq 1$ (we require 5 Newton iterations). Thus, $r \in \{q, q+1\}$ for $q = \lfloor \sqrt{m} \rfloor$.

Final Scaling and Check At this point, we have $r \in \{q, q+1\}$ for $q = \lfloor \sqrt{m} \rfloor$. This means

$$(r-1)^2 \leq m < (r+1)^2. \quad (\text{E.117})$$

We set

$$r = s2^e + t, \quad t \in \{0, 1, \dots, 2^{e-1}\}. \quad (\text{E.118})$$

This implies that

$$s = \left\lfloor \frac{r}{2^e} \right\rfloor. \quad (\text{E.119})$$

We see

$$\begin{aligned} (r-1)^2 &= (s2^e + t - 1)^2 \geq 2^{2e} (s-1)^2 \\ (r+1)^2 &= (s2^e + t + 1)^2 \leq 2^{2e} (s+1)^2. \end{aligned} \quad (\text{E.120})$$

After substituting $m = 2^{2e}n$ and using the previous inequalities, Eq. (E.117) reduces to

$$2^{2e} (s-1)^2 \leq 2^{2e}n < 2^{2e} (s+1)^2 \quad (\text{E.121})$$

or

$$(s-1)^2 \leq n < (s+1)^2. \quad (\text{E.122})$$

Thus, $s \in \{z, z+1\}$ for $z = \lfloor \sqrt{n} \rfloor$. The final check ensures we return the correct result. There is no overflow because we check if $s^2 \leq n$ by checking if $s \leq \lfloor n/s \rfloor$.

E.5.11 Hyper4

Initial Scaling See the discussion in Appendix E.5.10.

Error Bounds From the error bounds in Appendix E.3.6, we see that $\varepsilon_5 \leq 1$ (we require 5 Newton iterations). Thus, $r \in \{q, q+1\}$ for $q = \lfloor \sqrt{m} \rfloor$.

Final Scaling and Check See the discussion in Appendix E.5.10. This shows the correct result is returned.

E.5.12 Lookup4

Initial Scaling See the discussion in Appendix E.5.10.

Error Bounds From the error bounds in Appendix E.3.7, we see that $\varepsilon_5 \leq 1$ (we require 5 Newton iterations). Thus, $r \in \{q, q+1\}$ for $q = \lfloor \sqrt{m} \rfloor$.

Final Scaling and Check See the discussion in Appendix E.5.10. This shows the correct result is returned.

E.5.13 Lookup8

Initial Scaling See the discussion in Appendix E.5.10.

Error Bounds From the error bounds in Appendix E.3.8, we see that $\varepsilon_4 \leq 1$ (we require 4 Newton iterations). Thus, $r \in \{q, q + 1\}$ for $q = \lfloor \sqrt{m} \rfloor$.

Final Scaling and Check See the discussion in Appendix E.5.10. This shows the correct result is returned.

F Integer Square Root Computation with Floating-Point Square Root

We now investigate the problem of computing integer square roots with the assistance of *one* floating-point square root computation. While this is not possible in Solidity, this is possible for a general computer. This is mentioned in [5, Chapter 1.7, Remarks]: after suggesting using the largest power-of-2 strictly greater than $\sqrt{n}$ (the initialization value associated with `Unrolled2`), Cohen suggests that a second “option is to compute a single precision floating point approximation to the square root of n and to take the ceiling of that. The choice between these options is machine dependent.” Here, we will instead use *double precision* to obtain a more accurate initialization, but the idea is the same¹⁹.

We note that any *significant* computation involving large integers should use a dedicated library such as `GMP` [14] or `Flint` [9]; we note that `GMP` [14] references [39].

F.1 Compute Assumptions

Here, we focus on the situation similar to those of the rest of this work where we only have integer operations (addition, subtraction, multiplication, and integer division (with remainder)) while also being able to cheaply perform square roots in double precision²⁰. Floating-point computations have their own challenges and are worth studying; some references are [16] and [17, Chapter 4.2]. The primary metric we are concerned about is minimizing the number of Newton iterations; thus, we look to minimize the initial approximation error. We assume that all floating-point square roots are correctly rounded.

In double precision, the integers can be *represented exactly* (such as $\{0, 1, 2, \dots, 2^{53}\}$), while others cannot (such as $2^{53} + 1$); this is related to the fact that all `doubles` have 53 bits of precision²¹.

¹⁹And computing capabilities have significantly increased since 1993 when [5] was written.

²⁰Meaning, we treat square roots in double precision as having essentially zero cost when compared with the large integer operations also being performed.

²¹While 2^{53} is a 54-bit integer, it is still represented exactly as a `double`.

Algorithm 3 Integer Square Root algorithm using floating-point sqrt

Require: n is a positive integer

```
1: procedure INTEGER SQUARE ROOT( $n$ )
2:   if  $n \leq 2^{53}$  then                                ▷ All of these values are represented exactly using double
3:     return INT(SQRT( $n$ ))                                ▷ Compute the square root as a double and truncate
4:   end if
5:    $f := \text{BITLENGTH}(n)$ 
6:   if  $f \bmod 2 = 0$  then
7:      $\alpha = (f - 52)/2$ 
8:   else
9:      $\alpha = (f - 53)/2$ 
10:  end if
11:   $q := \lfloor n/2^{2\alpha} \rfloor$                                 ▷ Get the high bits of  $n$ 
12:   $w := q + 1$ 
13:   $\hat{w} := \text{INT}(\text{SQRT}(w)) + 1$ 
14:   $x := 2^\alpha \cdot \hat{w}$ 
15:   $y := \lfloor (x + n/x) / 2 \rfloor$ 
16:  while  $x > y$  do
17:     $x := y$ 
18:     $y := \lfloor (x + n/x) / 2 \rfloor$ 
19:  end while
20:  return  $x$ 
21: end procedure
```

F.2 Efficient Algorithms

See Algorithms 3, 4, and 5 for the implementations; Algorithm 3 follows closely the suggestion from [5, Chapter 1.7, Remarks]²², but Algorithms 4 and 5 provide more accurate initial values.

The main idea is the same: compute a floating-point square root of the high-order bits of n to get an initial approximation, and this is done in Algorithm 3. To obtain a more accurate initial value, Algorithm 4 scales the computed square root *before* truncating to an integer; this is seen in Line 15. This is possible because of the assumption that the computed square root is *correctly rounded*. Algorithm 5 is able to reach 52 bits of accuracy by the use of bitwise AND in Line 15 when determining the value to square root.

Provided the initial value x satisfies $x > \lfloor \sqrt{n} \rfloor$ (which is shown to hold for Algorithms 3 and Algorithms 4), the algorithms are provably correct (by previous work); because this is not necessarily the case for Algorithm 5, one additional Newton iteration is required before the **while** loop.

²²We truncate to an integer and add one rather than compute the ceiling, but this could easily be modified and would not affect the results.

Algorithm 4 Integer Square Root algorithm using floating-point sqrt (improved)

Require: n is a positive integer

```
1: procedure INTEGERSQUAREROOT( $n$ )
2:   if  $n \leq 2^{53}$  then                                 $\triangleright$  All of these values are represented exactly using double
3:     return INT(SQRT( $n$ ))                                $\triangleright$  Compute the square root as a double and truncate
4:   end if
5:    $f := \text{BITLENGTH}(n)$                                  $\triangleright$  We have  $2^{f-1} \leq n < 2^f$  and  $f \geq 54$ 
6:   if  $f \bmod 2 = 0$  then
7:      $\alpha = (f - 52)/2$ 
8:      $\beta = \min(\alpha, 28)$ 
9:   else
10:     $\alpha = (f - 53)/2$ 
11:     $\beta = \min(\alpha, 27)$ 
12:  end if
13:   $q := \lfloor n/2^{2\alpha} \rfloor$                                  $\triangleright$  Get the high bits of  $n$ 
14:   $w := q + 1$ 
15:   $\hat{w} := \text{INT}(2^\beta \cdot \text{SQRT}(w)) + 1$ 
16:   $x := 2^{\alpha-\beta} \cdot \hat{w}$ 
17:   $y := \lfloor (x + n/x) / 2 \rfloor$ 
18:  while  $x > y$  do
19:     $x := y$ 
20:     $y := \lfloor (x + n/x) / 2 \rfloor$ 
21:  end while
22:  return  $x$ 
23: end procedure
```

F.3 Proof of Initial Error Bound

We now prove initial error bounds; we focus on Algorithm 4 as the bound for Algorithm 3 follows when setting $\beta = 0$ while Algorithm 5 has only minor modifications. In particular, we have the bound

$$\begin{aligned} |\sqrt{n} - x| &\leq \begin{cases} 2^{e-52.5} + 2^{e-25-\beta} & \text{BITLENGTH}(n) \bmod 2 = 0 \\ 2^{e-53} + 2^{e-25.5-\beta} & \text{BITLENGTH}(n) \bmod 2 = 1 \end{cases} \\ &\leq 2^{e-51.7}, \quad n \geq 2^{106}. \end{aligned} \tag{F.1}$$

Following the work in Appendix E, we are assuming $2^{e-1} \leq \sqrt{n} < 2^e$. This shows that asymptotically Algorithm 4 has twice the accuracy of Algorithm 3. For Algorithm 5, we have the bound

$$\begin{aligned} |\sqrt{n} - x| &\leq \begin{cases} 2^{e-53.5} + 2^{e-25-\beta} & \text{BITLENGTH}(n) \bmod 2 = 0 \\ 2^{e-54} + 2^{e-25.5-\beta} & \text{BITLENGTH}(n) \bmod 2 = 1 \end{cases} \\ &\leq 2^{e-52}, \quad n \geq 2^{106}. \end{aligned} \tag{F.2}$$

Algorithm 5 Integer Square Root algorithm using floating-point sqrt (further improved)

Require: n is a positive integer

```

1: procedure INTEGER SQUARE ROOT( $n$ )
2:   if  $n \leq 2^{53}$  then                                 $\triangleright$  All of these values are represented exactly using double
3:     return INT(SQRT( $n$ ))                                $\triangleright$  Compute the square root as a double and truncate
4:   end if
5:    $f := \text{BITLENGTH}(n)$                                  $\triangleright$  We have  $2^{f-1} \leq n < 2^f$  and  $f \geq 54$ 
6:   if  $f \bmod 2 = 0$  then
7:      $\alpha = (f - 52)/2$ 
8:      $\beta = \min(\alpha, 28)$ 
9:   else
10:     $\alpha = (f - 53)/2$ 
11:     $\beta = \min(\alpha, 27)$ 
12:  end if
13:   $t := \lfloor n/2^{2\alpha+1} \rfloor$ 
14:   $q := \lfloor n/2^{2\alpha} \rfloor$                                  $\triangleright$  Get the high bits of  $n$ 
15:   $w := q + (1 \wedge t)$                                      $\triangleright \wedge$  is bitwise AND.
16:   $\hat{w} := \text{INT}(2^\beta \cdot \text{SQRT}(w)) + 1$ 
17:   $x := 2^{\alpha-\beta} \cdot \hat{w}$ 
18:   $x := \lfloor (x + n/x) / 2 \rfloor$                              $\triangleright$  This ensures  $x \geq \lfloor \sqrt{n} \rfloor$ 
19:   $y := \lfloor (x + n/x) / 2 \rfloor$ 
20:  while  $x > y$  do
21:     $x := y$ 
22:     $y := \lfloor (x + n/x) / 2 \rfloor$ 
23:  end while
24:  return  $x$ 
25: end procedure

```

Proof of Error Bound for Algorithms 3 and 4

We assume $2^{f-1} \leq n < 2^f$ and include important values from the algorithm:

$$\begin{aligned}
\alpha &:= \begin{cases} (f - 52)/2 & f \bmod 2 = 0 \\ (f - 53)/2 & f \bmod 2 = 1 \end{cases} \\
\beta_{\max} &:= \begin{cases} 28 & f \bmod 2 = 0 \\ 27 & f \bmod 2 = 1 \end{cases} \\
q &:= \left\lfloor \frac{n}{2^{2\alpha}} \right\rfloor \\
w &:= q + 1 \\
\bar{w} &:= 2^{2\alpha} w \\
s &:= \text{SQRT}(w) \quad (\text{computed value of } \sqrt{w}) \\
x &:= 2^{\alpha-\beta} (\lfloor 2^\beta s \rfloor + 1).
\end{aligned} \tag{F.3}$$

By construction, we have

$$\sqrt{n} < \sqrt{w}. \quad (\text{F.4})$$

We will also show that $\sqrt{w} < x$. Additionally, we have

$$2^{25} < w \leq 2^{27}. \quad (\text{F.5})$$

To prove the bound, we focus on

$$|\sqrt{n} - x| \leq |\sqrt{n} - \sqrt{w}| + |\sqrt{w} - x|. \quad (\text{F.6})$$

This inequality is actually equality which follows from Eq. (F.4) and the comment below (which will be proven).

We start by focusing on the first term. By the Mean Value Theorem (e.g., [28, Theorem 5.10]), there exists $\xi \in [v, \bar{w}]$ such that

$$|\sqrt{n} - \sqrt{w}| = \frac{1}{2\sqrt{\xi}} |n - \bar{w}|. \quad (\text{F.7})$$

The definition of $\bar{w}$ implies

$$|n - \bar{w}| \leq 2^{2\alpha}. \quad (\text{F.8})$$

The assumption on $\sqrt{n}$ implies $\xi \in [2^{f-1}, 2^f]$ so that

$$\frac{1}{2\sqrt{\xi}} \leq \frac{1}{\sqrt{2^{f+1}}}. \quad (\text{F.9})$$

Together, these imply

$$\frac{1}{2\sqrt{\xi}} |n - \bar{w}| \leq \begin{cases} 2^{e-52.5} & f \bmod 2 = 0 \\ 2^{e-53} & f \bmod 2 = 1 \end{cases}. \quad (\text{F.10})$$

We now focus on the second term. First, we see

$$|\sqrt{w} - x| = 2^{\alpha-\beta} |2^\beta \sqrt{w} - (\lfloor 2^\beta s \rfloor + 1)|. \quad (\text{F.11})$$

Because we have s is the computed square root of w which is *correctly rounded* in double precision and Eq. (F.5) we have

$$|\sqrt{w} - s| < 2^{-\beta_{\max}}. \quad (\text{F.12})$$

From this it follows that

$$2^{\beta_{\max}} \sqrt{w} < 2^{\beta_{\max}} s + 1. \quad (\text{F.13})$$

From Eqs. (F.12) and (F.13) we also obtain

$$2^\beta \sqrt{w} < \lfloor 2^\beta s \rfloor + 1 \quad (\text{F.14})$$

and

$$|2^\beta \sqrt{w} - (\lfloor 2^\beta s \rfloor + 1)| < 1 + 2^{\beta - \beta_{\max}} \leq 2. \quad (\text{F.15})$$

This brings us to

$$\begin{aligned} |\sqrt{w} - x| &\leq 2^{\alpha - \beta + 1} \\ &= \begin{cases} 2^{e-25-\beta} & f \bmod 2 = 0 \\ 2^{e-25.5-\beta} & f \bmod 2 = 1 \end{cases}. \end{aligned} \quad (\text{F.16})$$

Together this gives the desired result:

$$|\sqrt{n} - x| \leq \begin{cases} 2^{e-52.5} + 2^{e-25-\beta} & f \bmod 2 = 0 \\ 2^{e-53} + 2^{e-25.5-\beta} & f \bmod 2 = 1 \end{cases}. \quad (\text{F.17})$$

When $f \geq 107$ (equivalently, $n \geq 2^{106}$), the above inequality reduces to

$$\begin{aligned} |\sqrt{n} - x| &\leq 2^{e-52.5} + 2^{e-53} \\ &< 2^{e-51.7} \quad (n \geq 2^{106}). \end{aligned} \quad (\text{F.18})$$

The important part is that by using *one* double precision square root computation an initial value can be obtained with at least 51.7 bits of accuracy.

Proof of Error Bound for Algorithm 5

The proof for the error bounds for Algorithm 5 is essentially the same as the one for Algorithms 3 and 4. The only difference is the bound for w . Because we choose w based on if the next bit is set or not (this is the bitwise AND in Line 15), this implies

$$|n - \bar{w}| \leq 2^{2\alpha-1}. \quad (\text{F.19})$$

Thus, this first error term is slightly more accurate. Thus, we have

$$|\sqrt{n} - x| \leq \begin{cases} 2^{e-53.5} + 2^{e-25-\beta} & f \bmod 2 = 0 \\ 2^{e-54} + 2^{e-25.5-\beta} & f \bmod 2 = 1 \end{cases}. \quad (\text{F.20})$$

For large values of n , we see

$$\begin{aligned} |\sqrt{n} - x| &\leq \begin{cases} 2^{e-53.5} + 2^{e-53} & f \bmod 2 = 0 \\ 2^{e-54} + 2^{e-52.5} & f \bmod 2 = 1 \end{cases} \\ &< 2^{e-52} \quad (n \geq 2^{106}). \end{aligned} \quad (\text{F.21})$$

We have shown that it is possible to get 52 bits of accuracy by using *one* double precision square root computation; given that double precision numbers have 53 bits of precision, this

is essentially optimal²³. It is important to note that the “+1” in the initialization value x does not affect the error bound for the second error term; the error bound remains unchanged even if it is removed²⁴, but including it ensures that $x > \lfloor \sqrt{n} \rfloor$.

The author mentions the additional work in Algorithm 5 will likely make it inferior to Algorithm 4 for practical values; he expects the advantage of Algorithm 5 will be noticed by the user only when he should have long since switched to more efficient algorithms from GMP [14] or Flint [9].

²³It is not clear to the author how to *improve* this error bound using just *one* SQRT call. While the author thinks a slightly more accurate approximation could be obtained by averaging $\sqrt{q}$ and $\sqrt{q+1}$, he is not going to attempt to work through those details; see ²⁴.

²⁴This would appear to show that the second error term (the accuracy of the computed square root) limits the initial accuracy to at most 52.5 bits, and this is the main reason why the author is not attempting further improvements. Even if it were *possible* to perform a weighted average of $\text{SQRT}(q)$ and $\text{SQRT}(q+1)$ to obtain a better approximation, the author is not convinced the float-point error in the second term could be *proven* to decrease. With such a possible weighted value, the author could see how the first error term could be reduced to 2^{e-55} , but the second term would remain at $2^{e-52.5}$; the overall error would then reduce to $2^{e-52.26}$, which is a negligible improvement. The author is aware the difference between Algorithms 4 and 5 is negligible; Algorithm 5 was included only to show it is *possible* to get 52 bits of initial precision using one *double* square root (which has 53 bits).